DDoS Detection Lab

Lightweight Deep Learning (LUCID) And Incident Response Case Study

NAIRR Eager Project

October 17, 2026

1 DDoS Attacks and Flow-Based Detection

A **Distributed Denial of Service (DDoS)** attack tries to make a service unavailable by flooding it with more traffic than it can handle. Unlike a single-source attack, the traffic comes from many hosts at the same time, so simply blocking one address is rarely enough. Detecting the attack quickly, and telling attack traffic apart from legitimate users, is therefore one of the core tasks of a network defender.

1.1 Types of DDoS Attacks

DDoS attacks are usually grouped by the resource they exhaust:

- **Volumetric attacks:** Saturate the bandwidth of the victim's link with a very large number of packets (e.g., UDP or ICMP floods). In a **reflection attack**, such as DNS reflection, the attacker sends small requests with the victim's address as the sender to many servers, which all send their (much larger) answers to the victim.
- **Protocol attacks:** Exhaust the state tables of servers and firewalls. A **SYN flood** sends many TCP connection requests (SYN) and never completes the handshake, so the server keeps half-open connections until it runs out of resources.
- **Application-layer attacks:** Send requests that look legitimate, such as an **HTTP flood** that requests web pages in parallel loops, to exhaust the application itself.

The dataset of this lab combines two networks (Section 3.5): the CIC-DDoS2019 sample that the LUCID authors publish with their code, with DNS reflection, SYN flood, UDP-Lag (a UDP flood that slows down online games) and Web DDoS attacks, and a recording of the lab network with SYN floods and HTTP floods against a web server, mixed with the traffic of 17 legitimate users.

1.2 Effects of DDoS Attacks

Whatever resource is targeted, the result is that legitimate users can no longer be served:

- **Loss of availability:** Pages load slowly or not at all, connections time out, and online services such as shops, banking or government portals stop working.
- **Exhausted resources:** A SYN flood fills the server's table of half-open connections, so new legitimate connections are refused; an HTTP flood keeps the web application busy with useless work; a volumetric flood fills the link itself.
- **Collateral damage:** Other services that share the same link, server or provider are slowed down or cut off as well.
- **Cost and reputation:** Downtime means lost revenue and customer trust, and mitigation (extra bandwidth, scrubbing services, staff time) costs money.
- **Distraction:** A DDoS attack can keep the security team busy while another attack, such as a data theft, takes place.

1.3 Classical Detection and Its Limitations

Classical DDoS defences do not learn from data. They rely on rules written by experts:

- **Threshold (rate) based detection:** Traffic is counted per source address (or per destination) in a time window, and an alarm is raised when a counter passes a fixed limit, for example "more than 10 SYN packets per second from one address".
- **Signature based detection:** Packets are compared with patterns of known attack tools (e.g., a specific payload or header combination).
- **Statistical anomaly detection:** The current traffic is compared with a "normal" baseline, e.g. the usual number of connections at this time of day.

These methods are fast and easy to explain, but they have well-known limitations:

- **Thresholds must be tuned for every network.** Too low, and busy legitimate users or a flash crowd are blocked; too high, and the attack passes.
- **Distributed attacks stay under per-source limits.** A botnet spreads the same attack over thousands of addresses, so every single source looks harmless.
- **Application-layer attacks look legitimate.** An HTTP flood consists of valid requests; a rule that only counts packets cannot tell them from real visitors.
- **Signatures only know yesterday's attacks.** New tools or slightly changed packets are not recognised.

In this lab you implement a classical threshold detector yourself, tune it on the training data and see exactly where it breaks (Experiment 4).

1.4 Why Machine Learning?

A machine learning model can instead learn *what attack traffic looks like* from labelled examples and judge every flow individually, without fixed limits and without looking at who sends the traffic.

LUCID (Lightweight, Usable CNN in DDoS Detection) is a deep learning approach designed for this task. It represents the traffic of every flow as a small matrix of packet header features and classifies it with a very small Convolutional Neural Network (CNN), so it can run on resource constrained devices and still decide within seconds.

1.5 Capturing Packets in Time Windows

Network traffic is a continuous stream. An online intrusion detection system cannot wait until a flow has ended: it collects the traffic for a certain amount of time and then has to decide on the part of each flow it has seen so far. LUCID simulates exactly this capturing process with **time windows** of t seconds [1]:

- A window starts at a packet and collects every packet that arrives within the next t seconds (in the paper and in this lab, $t = 100$ s).
- The next window starts at the *first packet that arrives after* the previous window ended, no matter how long the network was silent in between. If no packet arrives for 400 seconds, the next window simply starts with the next packet (Figure 1a).
- Every window gives one detection decision, so an attack is reported at the latest one window after it starts.

Why not process every packet? Running a model on every single packet would be expensive and unnecessary. Working window by window keeps the memory bounded, and inside a window only the first n packets of every flow are kept (Section 2.3). In this lab, 1.9 million captured packets become 45,504 balanced samples of at most 100 packets each. Selecting packets this way reduces the work while keeping the part of every flow that shows how it behaves.

Why 100 seconds? The window length is a trade-off. Short windows allow an attack to be detected within a very short time, while longer windows give the model more information about every flow and therefore higher accuracy. The LUCID authors tested windows from 1 to 100 seconds in a grid search and selected $t = 100$ s together with $n = 100$ packets, the configuration with the best F1 score on their validation data; they also observed that the CNN is relatively insensitive to t once more than one packet per flow is kept [1].

How botnets attack. DDoS attacks are launched by botnets, large groups of compromised hosts controlled by an attacker. A study of 50,704 attacks launched by 674 botnets of 23 families over seven months found that repeated attacks on the same target follow strong patterns in the time between attacks, and that different botnets often attack the same victim simultaneously or in turn [2]. Attacks also come in short, intense bursts: the largest attack reported for 2025 reached 31.4 Tbps and lasted only 35 seconds [3]. A detector therefore has to decide within seconds to minutes, which is why the window, and not the whole recording, is the unit of decision.

1.6 Bidirectional Flows and Labels

Inside every window, packets are grouped into **flows** by their 5-tuple (IP A, port A, IP B, port B, protocol). Both directions of a conversation belong to the same flow, so a request and its answer are seen together. One (window, flow) pair is one **sample** (Figure 1b).

- **Label:** A flow between an attacker address and the victim address is labelled DDoS (1); every other flow is benign (0).
- **Balance:** The training data contains as many DDoS samples as benign samples, so the model cannot score well by always predicting the larger class.
- **Split by flow:** 90 % of the flows are used for training and 10 % for testing; 10 % of the training flows are kept as a validation split. All windows of one flow go to the same split, so no flow is seen in both training and testing.

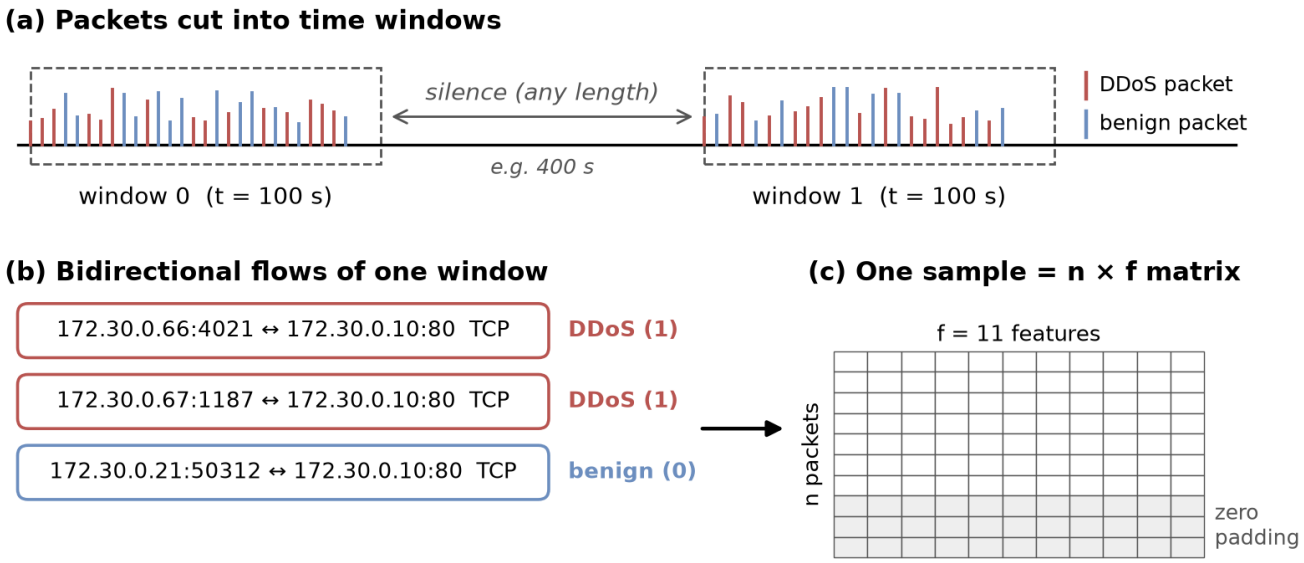


Figure 1: From packets to samples: time windows, bidirectional flows and the $n \times f$ sample matrix

2 Features, Shaping and Normalization

Before a CNN can learn from network traffic, every flow sample has to be described by numbers, brought into a fixed shape and scaled to a common range. This section explains which packet features LUCID uses and why identifiers such as IP addresses are left out, how variable-length flows become fixed-size matrices, and how the four normalization options of this lab differ.

2.1 Packet Features

Every packet of a flow is described by the 11 header features used by LUCID (Table 1). The 5-tuple is only used to group packets into flows; it is *not* a model input.

Feature	Meaning
`time`	Arrival time of the packet relative to the first packet of its flow
`packet_len`	Length of the IP packet in bytes
`highest_layer`	Highest protocol layer found in the packet (encoded)
`ip_flags`	IP header flags (e.g., Don't Fragment)
`protocols`	Protocols present in the packet (bit mask: tcp, udp, http, dns, ...)
`tcp_len`	Length of the TCP payload
`tcp_ack`	TCP acknowledgement number
`tcp_flags`	TCP flags (SYN, ACK, FIN, RST, ...)
`tcp_win`	TCP window size
`udp_len`	Length of the UDP datagram
`icmp_type`	ICMP message type

Table 1: The 11 LUCID packet features

2.2 Feature Selection and Shortcut Learning

LUCID deliberately leaves out IP addresses and ports: *"We intuitively exclude those attributes that would be detrimental to the generalization of the model, such as IP addresses and TCP/UDP ports"* (Doriguzzi-Corin et al., 2020). In a recorded dataset the attacker always uses the same address, so a model that sees addresses can score perfectly on the test split by simply **memorising the attacker's IP**. This is called **shortcut learning**: the model learns an easy rule that holds in the dataset but not in the real world, and it fails as soon as the attacker comes from another address.

To make this visible, the lab builds a second test split, `test_newaddr`, with the same test flows but the attacker and benign client addresses swapped (as if the attacker had taken over a legitimate workstation). Three feature sets are compared:

- **lucid:** The 11 LUCID features (no identifiers).
- **lucid_ids:** The 11 features plus source/destination IP and port.
- **ids_only:** Only the identifiers. This shows how far a model gets by memorising addresses.

This experiment runs on the lab network only, the network you defend in the post-lab. Min-max scaling turns an address into a number between 0 and 1; fitted on both networks, whose addresses range from `172.16.x.x` to `192.168.x.x`, it would make all addresses of the lab network almost the same number, and the shortcut could not be seen.

2.3 Shaping: The $n \times f$ Sample Matrix

A CNN needs inputs of a fixed size, but flows have very different lengths. Every sample is therefore shaped into an $n \times f$ matrix: one row per packet, one column per feature (Figure 1c). In the paper, $n = 100$.

- The first n packets of the flow are kept in time order.
- A flow with fewer packets is **padded** with rows of zeros.
- A longer flow is **cut** after n packets; the remaining packets are never seen by the model.

A small n saves memory and time but may cut away exactly the packets that show the attack. The shaping experiment compares $n = 3, 10, 50$ and 100.

2.4 Normalization

Raw header values live on very different scales: a TCP acknowledgement number can reach $2^{32} \approx 4 \cdot 10^9$, while TCP flags stay below 30. Without scaling, the features with large values dominate the gradients and training becomes unstable (**feature dominance**). Four options are compared in this lab:

- **none:** Raw header values are fed to the network.
- **theoretical** (LUCID's choice): Min-max scaling with the *theoretical* range of every field, e.g. window size $0 \ldots 65535$, acknowledgement $0 \ldots 2^{32}$, time $0 \ldots t$:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

  The same scale is valid for any network, so no data is needed to fit it (a "super-global" normalization). Real values often use only a small part of the range.

- **global:** The same min-max formula, but $x_{\min}$ and $x_{\max}$ are measured on the *training* data and then applied unchanged to the validation and test data. It uses the full $[0, 1]$ range, but new traffic can produce values outside the training range.
- **batchnorm:** Values stay raw in the data; a **Batch Normalization** layer in front of the CNN learns the scaling during training:

$$\hat{x} = \gamma \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta$$

  where μ_B and σ_B^2 are the mean and variance of the batch, and γ, β are learned.

Important: The scaling parameters are always fitted on the training split only. Using the test data to fit them would leak information about the test set into training.

3 Model Architecture

This section introduces the model that classifies the samples. It first explains how a Convolutional Neural Network works in general, then describes the small CNN used by LUCID, the fixed settings it is trained with in this lab, and how its results are evaluated with confidence intervals.

3.1 What Is a Convolutional Neural Network?

A **Convolutional Neural Network (CNN)** is a neural network that learns small pattern detectors, called **filters** or **kernels**, instead of one weight per input value.

- **Convolution:** A filter is a small matrix of weights. It slides over the input and, at every position, multiplies its weights with the values below it and adds them up. The result is a **feature map** that shows where in the input the filter's pattern appears. Because the same filter is used at every position, a pattern is recognised wherever it occurs.
- **Activation:** A non-linear function such as ReLU ($\max(0, x)$) is applied to the feature map, so the network can learn more than linear combinations.
- **Pooling:** Pooling summarises a feature map, for example by keeping only its maximum, which makes the result independent of where exactly the pattern appeared.
- **Dense (fully connected) layer:** The pooled values are combined into the final decision, here the probability that the input is a DDoS flow.
- **Training:** All weights start at random values. For every batch of training samples the network's error (the *loss*) is computed, and the weights are changed a little in the direction that reduces it (back-propagation with the Adam optimizer). One pass over the whole training set is one **epoch**.

CNNs are best known from images, but they work on any input where neighbouring values belong together. In a flow sample, neighbouring rows are consecutive packets, so a filter can learn patterns such as "a SYN, then a SYN-ACK, then a reset" that are typical for an attack.

3.2 The LUCID CNN

LUCID uses a deliberately small 2D CNN (Figure 2):

- **Convolution:** $k = 64$ kernels of size $h \times f = 3 \times f$. Because the kernel is as wide as the matrix, it slides only along the packet axis and looks at three consecutive packets at a time. The order of the rows therefore matters.
- **ReLU activation** followed by **global max pooling**: for every kernel, only its strongest response anywhere in the flow is kept, which gives a vector of 64 values.
- **Dense layer with sigmoid:** One output neuron gives the probability $p(\text{DDoS})$; a sample is classified as DDoS when $p \geq 0.5$.

With $f = 11$ features the whole network has only $64 \cdot (3 \cdot 11) + 64 + 64 + 1 = 2241$ trainable parameters, which is why it trains in about a minute on a single CPU.

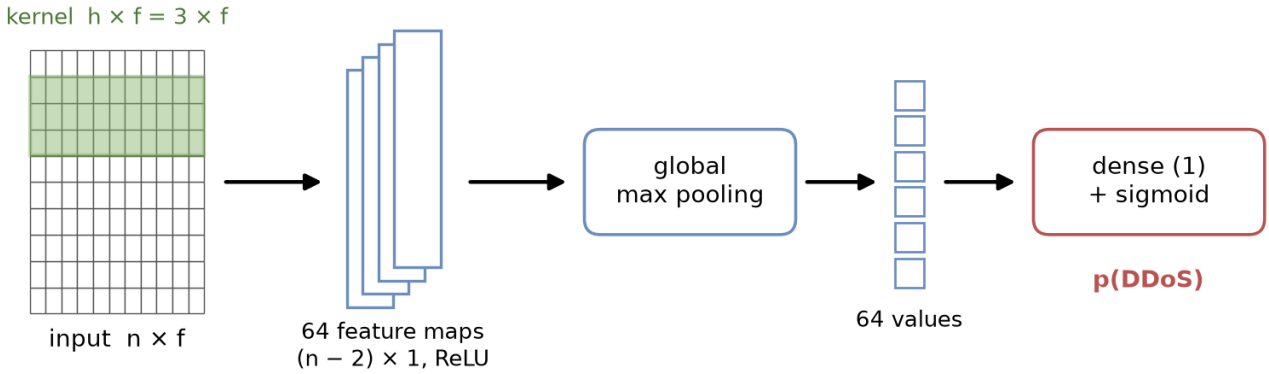


Figure 2: The LUCID CNN: convolution over the packet axis, global max pooling, dense output

3.3 Training Settings

The settings are fixed, so every student gets the same result:

- **Optimizer:** Adam with learning rate 0.01 (as in the paper), batch size 64, binary cross-entropy loss.
- **Epochs:** 10, which fits a lab session on one virtual CPU. No hyper-parameter search is done, so the validation split is not needed for training; it is kept aside for the live attack in the post-lab.
- **Reproducibility:** Seed 42, one CPU thread and deterministic algorithms. The same data always gives the same model on the same computer; on another computer the rounding of a few operations can differ, which changes the result of two unstable runs (Experiments 1 and 6).

3.4 Evaluation and Confidence Intervals

Predictions on the test split are compared with the true labels:

- **Accuracy:** Share of all samples classified correctly.

- **Precision:** Of the samples flagged as DDoS, how many really were DDoS.
- **Recall (TPR):** Of the real DDoS samples, how many were caught.
- **F1:** Harmonic mean of precision and recall (the paper's main metric).
- **False-Positive Rate (FPR):** Share of benign samples wrongly flagged as DDoS.

A single number does not show how certain it is. The lab therefore reports **95 % bootstrap confidence intervals**: the test samples are resampled with replacement 1000 times, the metric is recomputed each time, and the interval holds the middle 95 % of the values. Two runs whose intervals overlap cannot be called different with confidence.

3.5 Additional References and Dataset Information

For more detailed information about LUCID, refer to:
LUCID: A Practical, Lightweight Deep Learning Solution for DDoS Attack Detection
R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. Martínez-del-Rincón, D. Siracusa, IEEE Transactions on Network and Service Management, vol. 17, no. 2, 2020.
Available at: `https://doi.org/10.1109/TNSM.2020.2971776`

Datasets of the LUCID paper:
LUCID was developed and evaluated on three public datasets of the Canadian Institute for Cybersecurity (University of New Brunswick): ISCX2012 [4], CIC-IDS2017 [5] and CSE-CIC-IDS2018 [6]. They contain several days of benign and malicious traffic as packet captures with flow labels; the parts used by LUCID hold a DDoS attack of an IRC botnet (ISCX2012) and HTTP floods generated with the tools LOIC (CIC-IDS2017) and HOIC (CSE-CIC-IDS2018) [1]. The authors merged and balanced them into one dataset, called UNB201X in the paper.

Dataset Used in This Lab:
The lab uses the data of LUCID itself and adds a recording of the lab network to it. Both captures are read together and labelled with the same rule as in LUCID (a flow between an attacker and a victim is DDoS, Section 1.6):

- **CIC-DDoS2019 sample** (`data/cic_ddos2019_sample.pcap.gz`): the sample traffic that the LUCID authors publish with their code [7], taken from the CIC-DDoS2019 dataset of the Canadian Institute for Cybersecurity [8]. It contains benign traffic and four attacks: DNS reflection, SYN flood, UDP-Lag and Web DDoS, all from `172.16.0.5` against `192.168.50.1` and `192.168.50.4` (80,100 packets, 18,556 samples).
- **Lab network** (`data/lab_traffic.pcap.gz`): recorded for this lab in an isolated container network with the addresses of the post-lab network. Six sessions (2 benign, 2 SYN flood, 2 HTTP flood), each about 185 s long (60 s benign, 60 s attack on top of the benign traffic, 60 s benign) and separated by 400 s of silence. 17 benign users with different behaviour (browsing, API calls, keep-alive connections, downloads and slow readers); the attacks (a SYN flood sent with `hping3` and an HTTP flood of ten parallel request loops) come from `172.30.0.66` against the web server `172.30.0.10` (1.8 million packets, 135,154 samples).
- **Together:** 1.9 million packets and 153,710 samples; the balanced set has 45,504 samples (train 36,858, validation 4,095, test 4,551, of which 447 come from the CIC-DDoS2019 sample and 4,104 from the lab network).

The second capture makes the dataset fit the post-lab: the model you train has seen traffic of the network it defends. How much this matters is the subject of Experiment 6. The source and licence of the CIC-DDoS2019 sample are in `data/README.txt`; the recording scripts of the lab network are part of the lab repository (`tools/record_lab_sessions.sh` and `tools/build_lab_dataset.sh`).

4 Case Study: Live DDoS Incident Response

This section turns the theory into a realistic incident. After building and evaluating your own detector in the lab part, you use it in the post-lab to defend a web server during a live attack.

4.1 Scenario Background: A Web Server Under Attack

You work in the Security Operations Center (SOC) of a company whose public web server (`172.30.0.10`) has suffered short outages. The monitoring team suspects a DDoS attack hidden in the normal customer traffic.

Situation Overview:

- **Suspicious Activity:** Two hosts on the network (`attacker1` and `attacker2`) are suspected of flooding the server, while a legitimate client keeps using it.
- **Available Data:** A labelled recording of earlier traffic is available for training. The attack you will face is built from the *validation* split, traffic your model has never seen.
- **Constraint:** Legitimate users must not be cut off. Blocking the wrong host is also an incident.

Figure 3 shows the network of the post-lab.

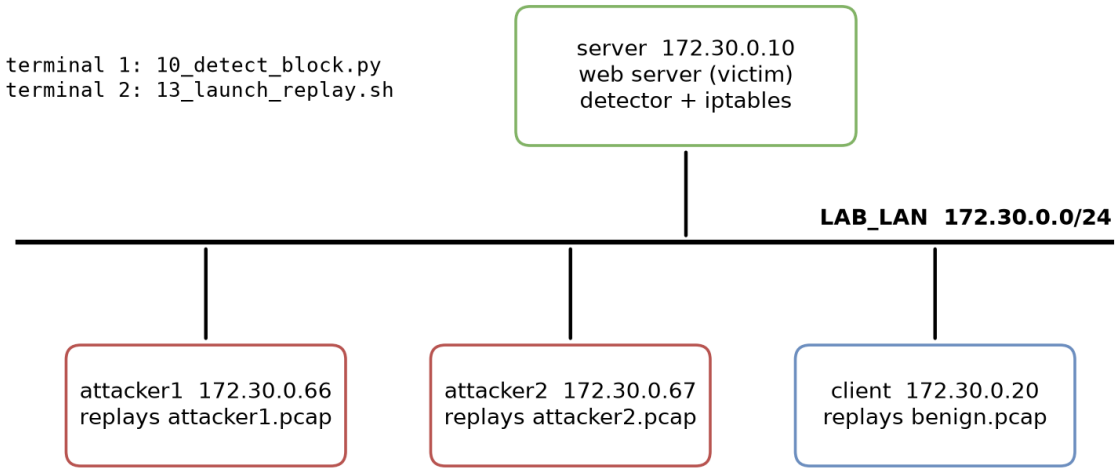


Figure 3: Post-lab network: the server defends itself against two attackers while a client keeps working

4.2 Task Definition and Objectives

As the analyst on duty, your responsibilities include:

- **Live Detection:** Run your trained model on the server's interface and classify every flow.
- **Containment:** Block the attacking addresses with firewall (iptables) rules while the attack is still going on, without blocking the legitimate client.
- **Post-Incident Reporting:** Produce a report with the attack window, the attacking addresses, the model's confidence (with confidence intervals), the detection quality and the time it took to block each attacker.

4.3 Analysis Methodology

The workflow follows these steps:

1. **Preparation:** Set up SSH access from the server to the traffic containers and build the replay files from the validation split.
2. **Detection and Blocking:** Start the detector on the server (terminal 1).
3. **Attack:** Start the attack and the benign traffic from the three containers (terminal 2).
4. **Reporting:** Turn the detection log into a post-incident report.

4.4 Learning Outcomes

Upon successfully completing this lab, participants will be able to:

- Explain the types of DDoS attacks and their effects.
- Process network traffic in the form required by the LUCID model (time windows, flows, normalization, feature selection, shaping).
- Analyse the limitations of classical DDoS detection methods.
- Apply a LUCID-based DDoS detection model.
- Compare classical and AI-based DDoS detection methods.
- Evaluate the decisions and limitations of an AI-based DDoS detection model.
- Develop a live DDoS detection and prevention solution.

5 Laboratory Setup and Preparation

This laboratory runs inside the Labtainer virtual machine. All required dependencies (PyTorch, NumPy, Matplotlib, tshark, iptables, tcpreplay), the scripts and the dataset are provided in preconfigured Docker images, so no additional setup is needed.

5.1 Setting Up the Laboratory Environment

First, ensure that you are operating in **student mode** inside the Labtainer virtual machine. Open a terminal and navigate to the following directory (Figure 4):

```
cd ~/labtainer/labtainer-student
```

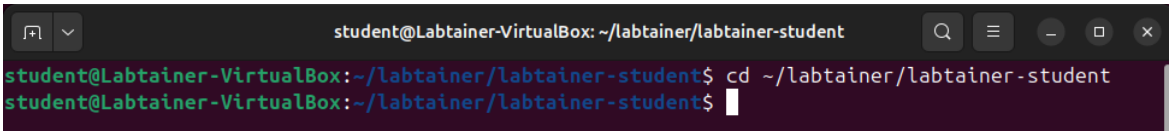


Figure 4: Terminal directory view

5.2 Installing the Lab Module

Add the lab to Labtainer with the following command (the backslash continues the command on the next line); Figure 5 shows the expected output:

```
imodule \
  https://github.com/Berkaybbayramoglu/DDoS_Lab/releases/download/imodule/imodule.tar
```

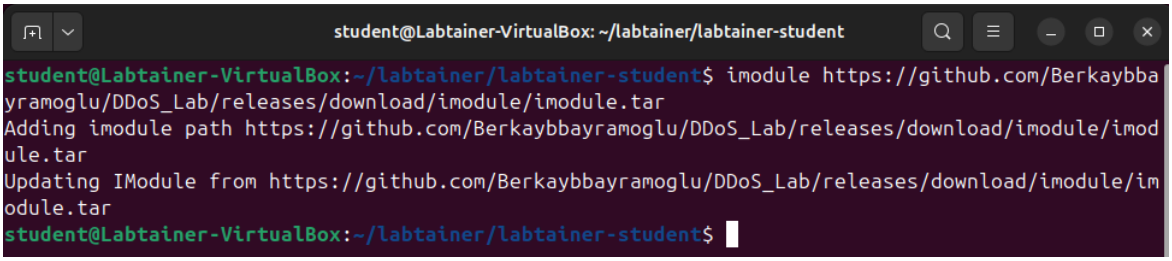


Figure 5: Module installation

This command adds the lab definition. The Docker images with all components are downloaded when the lab is started for the first time.

5.3 Starting the Laboratory

Start the lab with:

```
labtainer lucid-ddos
```

On the first start, the four container images (`server`, `attacker1`, `attacker2`, `client`) are downloaded from Docker Hub.
Important Note: The images need about 3 GB of disk space; the download takes a few minutes depending on your internet connection (Figure 6). At the end, the system asks for your e-mail address, which identifies your work for grading (Figure 7).

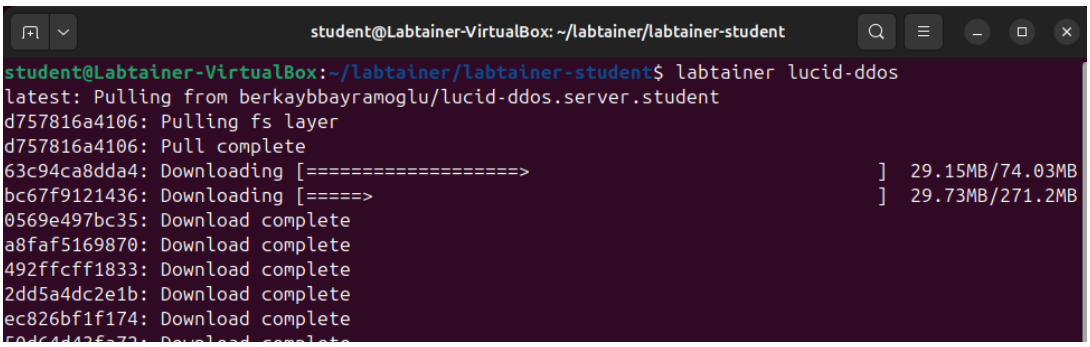


Figure 6: Downloading the lab images

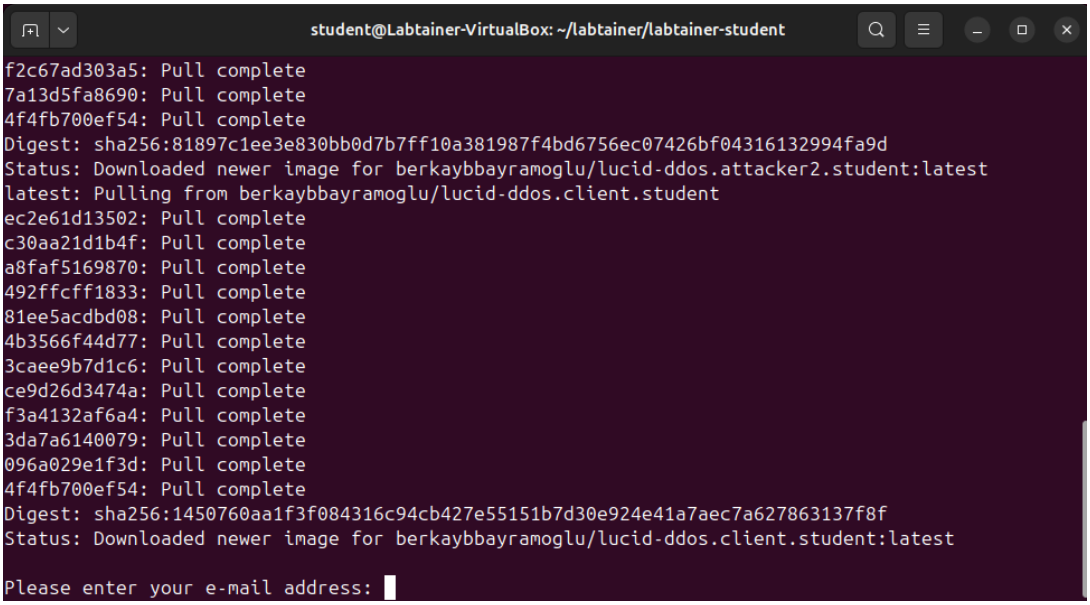


Figure 7: E-mail address prompt

When the lab is ready, Labtainer shows where the lab manual is (Figure 8). Press Enter to open the terminals.

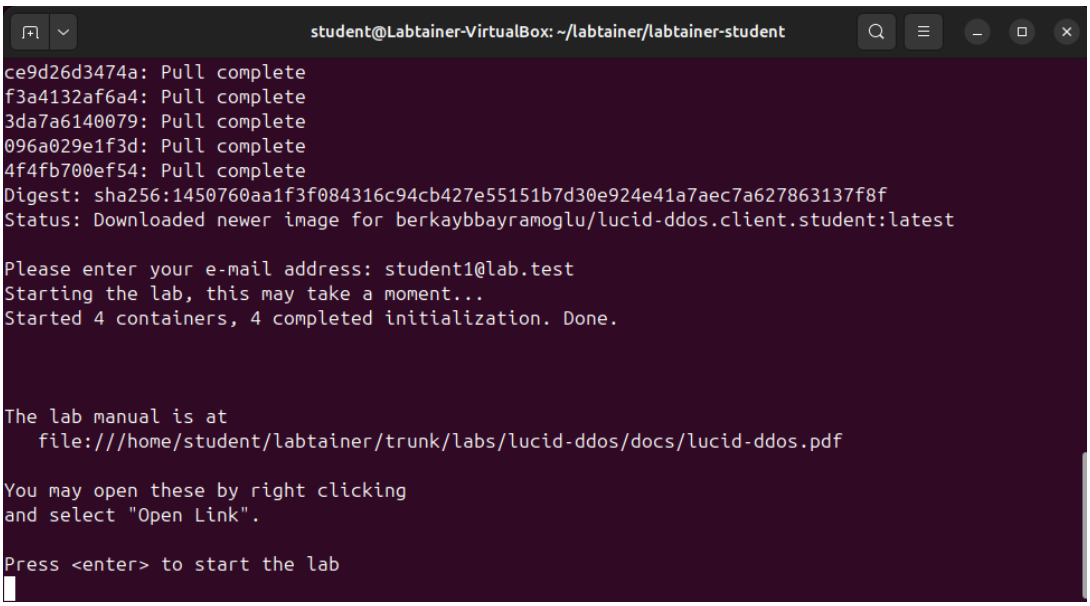


Figure 8: The lab has started

Two terminals open on the `server` container (`ubuntu@server`). You work in the first one; the second one is needed in the post-lab. The other containers have no terminal; you reach them over SSH in the post-lab.
Troubleshooting Tip: If you encounter errors or want to start again from a clean container, use the following command (all previous work in the lab is deleted):

```
labtainer -r lucid-ddos
```

5.4 Updating to a New Version of the Lab

Labtainer does not refresh container images it has already downloaded. If a new version of the lab is published after you first started it, update it from the Labtainer terminal (`~/labtainer/labtainer-student`):

```
stoplab lucid-ddos
for c in server attacker1 attacker2 client; do \
    docker pull berkaybbayramoglu/lucid-ddos.$c.student; done
labtainer -r lucid-ddos
```

`stoplab` first saves your current work in the `labtainer_xfer` folder, the `docker pull` commands download the new images, and `-r` starts the lab from fresh containers built from them. **Important:** `-r` deletes all previous work inside the lab, so update before you start working, or keep the saved results from `labtainer_xfer`.

5.5 Directory Structure and Files

Once the laboratory environment is initialized, inspect the working directory on the server (Figure 9):

```
pwd; ls -l
```

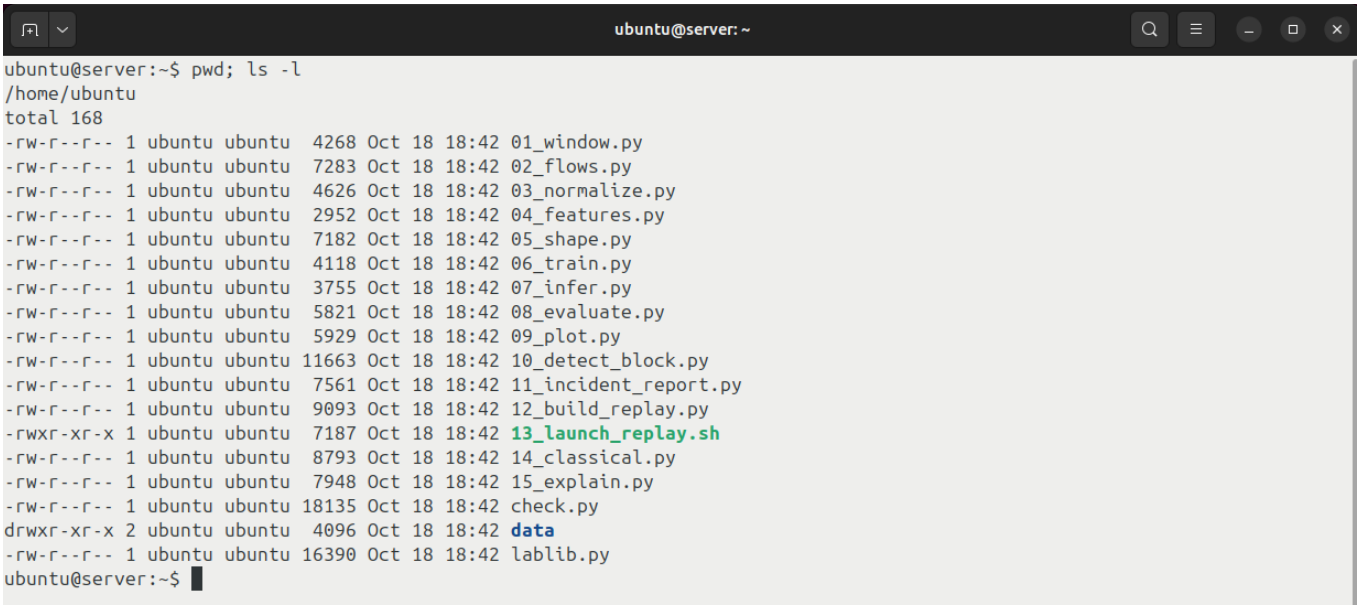


Figure 9: Contents of the home directory on the server

Each script covers one learning outcome (Table 2). It writes its result into a folder (with a `pipeline.json` that records its settings), which the next script reads.

File	Description
`data/cic_ddos2019_sample.pcap.gz`	The CIC-DDoS2019 sample of the LUCID authors (80,100 packets, compressed)
`data/lab_traffic.pcap.gz`	The recording of the lab network (1.8 million packets, compressed)
`data/README.txt`	Source, citation and licence of the two captures
`01_window.py`	Reads the capture and cuts it into time windows of t seconds
`02_flows.py`	Groups packets into bidirectional flows, labels and balances them, splits train/val/test
`03_normalize.py`	Scales the features: `none`, `theoretical`, `global` or `batchnorm`
`04_features.py`	Selects the features the model may see
`05_shape.py`	Builds the $n \times f$ matrix of every sample (padding / cutting)
`06_train.py`	Trains the LUCID CNN
`07_infer.py`	Applies a saved model to a data split
`08_evaluate.py`	Computes the metrics with 95 % bootstrap confidence intervals
`09_plot.py`	Draws one comparison figure per experiment
`14_classical.py`	A classical threshold detector, for comparison with LUCID
`15_explain.py`	Shows which features a model relies on and where it fails
`check.py`	Checks the output of a preprocessing step against an independent re-computation
`10_detect_block.py`	Post-lab: live detection and blocking on the server interface
`11_incident_report.py`	Post-lab: post-incident report from the detection log
`12_build_replay.py`	Post-lab: builds the attack and benign replay files from the validation split
`13_launch_replay.sh`	Post-lab: starts the replay on attacker1, attacker2 and client
`lablib.py`	Shared functions used by all scripts

Table 2: Files in the server's home directory

Tip: Every script explains its options with `--help`, for example `python3 05_shape.py --help`.

6 Building the Dataset

In this phase you turn the raw capture into training samples step by step. After each step, run `check.py`: it recomputes the rules of the step independently and shows whether every rule holds. **Run every command in the server terminal**, since your results are collected from there.

Reading the check output. `check.py` prints one table row per rule (see Figure 11):

- **rule:** what is checked, e.g. "every packet within [window start, start + t]".
- **result:** how many items satisfy the rule out of how many were checked, e.g. `1885744/1885744`.
- **PASS / FAIL:** `PASS` when every item satisfies the rule.

The last line gives the score of the step as a percentage, for example `window stage - 100 % of the rules pass`. Every step must reach **100 %** before you go on, because the next step builds on its output. If a rule shows `FAIL`, compare your command with the one in this manual (input folder, `-t`, `-n`, `--mode` or `--set`), run the step again and repeat the check. The models and experiments are checked in the same way later: `08_evaluate.py` shows the metrics of every run, and at the end `check.py --stage results` compares all of your runs with the reference results (Section 8.7).

Progress check. After each step you can also run `checkwork lucid-ddos` in the Labtainer terminal (Section 12): the goal of the step, named in the note after the step, then shows `Y`.

Your results. Every script also saves its result in a file; Section 8.9 lists where, and shows how to look at the files.

Re-running a step. Every step records in its `pipeline.json` which output of the previous step it was built from. If you run an earlier step again with other settings, the later steps stop with a message such as *"... changed after this pipeline was built from it (an earlier step was re-run). Run 03_normalize.py, 04_features.py and 05_shape.py again."* Run the steps it names again, in order. In the same way, `07_infer.py` refuses to apply a model to data that was prepared differently (another window, normalization, feature set or n), so a model and its data always belong together.

6.1 Step 1: Time Windows

```
python3 01_window.py --pcap data/cic_ddos2019_sample.pcap.gz \
    data/lab_traffic.pcap.gz -t 100 --out work/01_window
python3 check.py --stage window --dir work/01_window
```

Reading the 1.9 million packets of the two captures takes about one minute; Figure 10 shows the output.

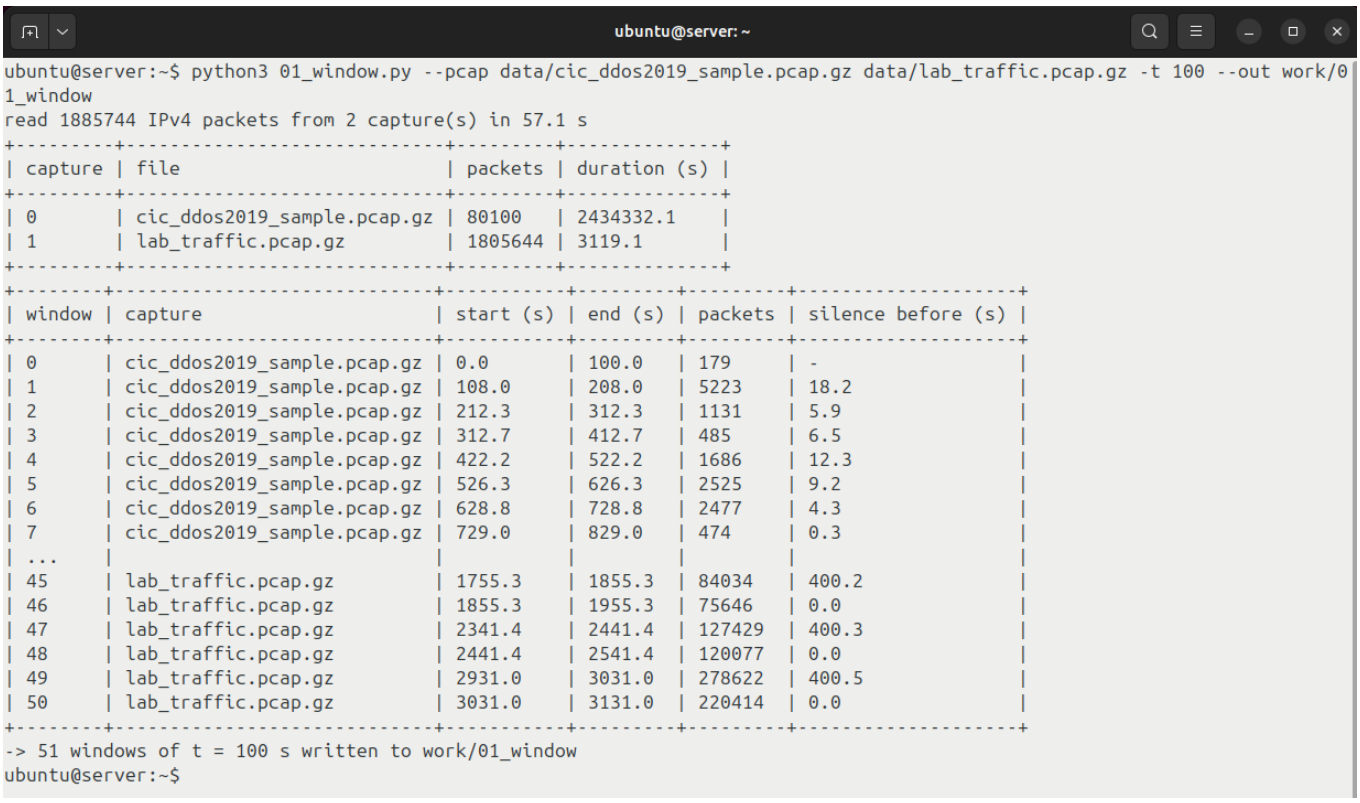


Figure 10: Windows of $t = 100$ s cut from the capture

The first table in Figure 10 lists the two captures; the second one lists the windows (the first and the last ones) with the capture they come from, their start and end within that capture and their number of packets. The packets of both captures are put in time order and cut into windows in the same way. The CIC-DDoS2019 sample consists of short traces recorded on different days, so its *duration* spans several weeks, but only the recorded traffic ends up in windows. The column *silence before* shows the gaps between the recording sessions: the next window does not start where the previous one ended but at the first packet after the silence. The check of this step must pass every rule (Figure 11).

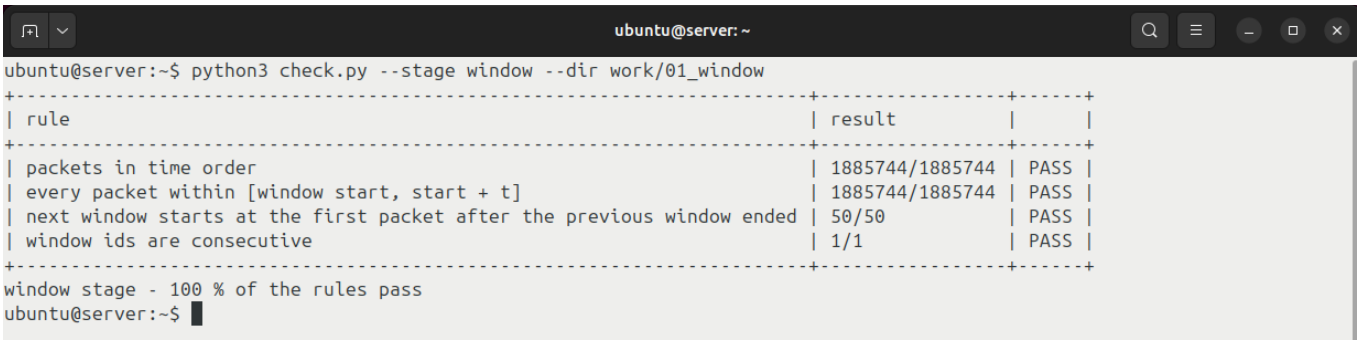


Figure 11: Checking the window step

Progress check: this step completes the `checkwork` goal `window_check` (Section 12).

6.2 Step 2: Flows, Labels and Splits

```
python3 02_flows.py --in work/01_window \
    --attackers 172.16.0.5 172.30.0.66 172.30.0.67 \
```

```
    --victims 192.168.50.1 192.168.50.4 172.30.0.10 --out work/02_flows
python3 check.py --stage flows --dir work/02_flows
```

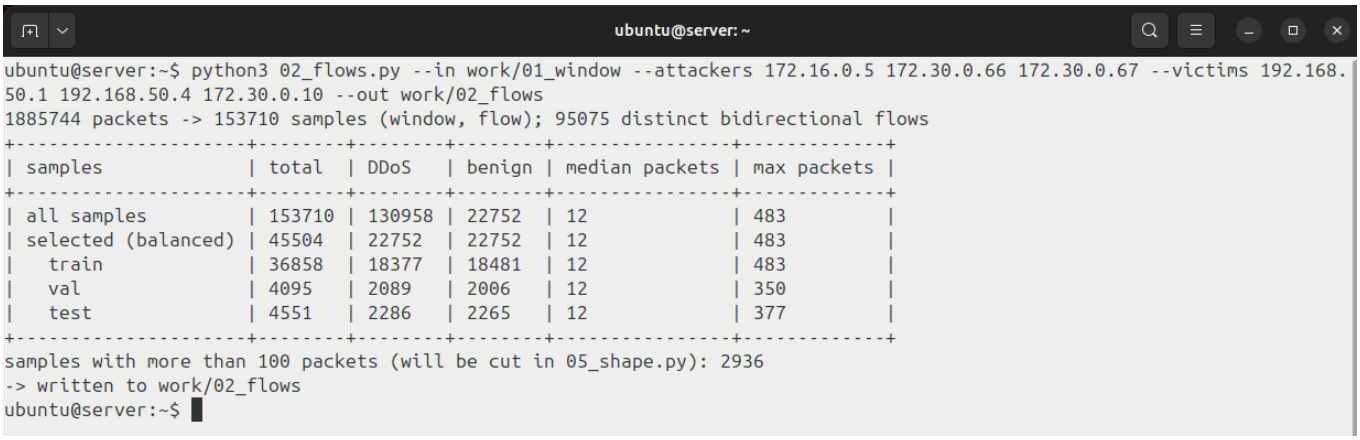


Figure 12: Flows, labels and the balanced train/val/test split

The attackers and victims of both networks are given together: `172.16.0.5` attacks `192.168.50.1` and `192.168.50.4` in the CIC-DDoS2019 sample, `172.30.0.66` and `172.30.0.67` attack `172.30.0.10` in the lab network. As Figure 12 shows, 130,958 of the 153,710 samples are DDoS and only 22,752 benign. After balancing, 45,504 samples remain (half DDoS, half benign), split into train, val and test. The check of the flow step is shown in Figure 13. The shares of the split follow the paper and can be changed with `--test` and `--val` (both 0.10 by default); `--no_balance` keeps every sample instead of balancing the two classes. Use the defaults for the lab, because the reference results (Section 8.7) are computed with them.

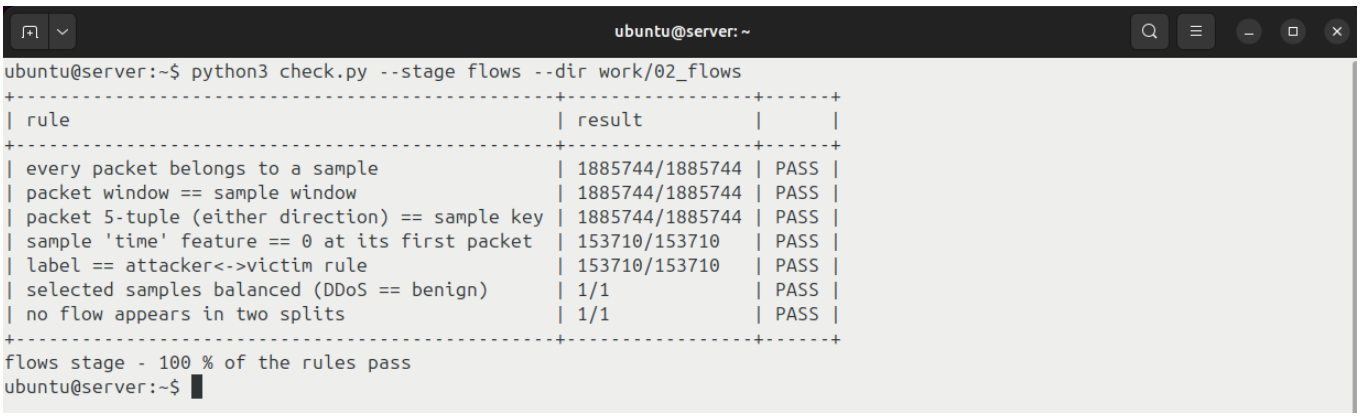


Figure 13: Checking the flow step

Progress check: this step completes the `checkwork` goal `flows_check` (Section 12).

6.3 Step 3: Normalization

Start with the `global` mode:

```
python3 03_normalize.py --in work/02_flows --mode global
python3 check.py --stage normalize --dir work/03_norm_global
```

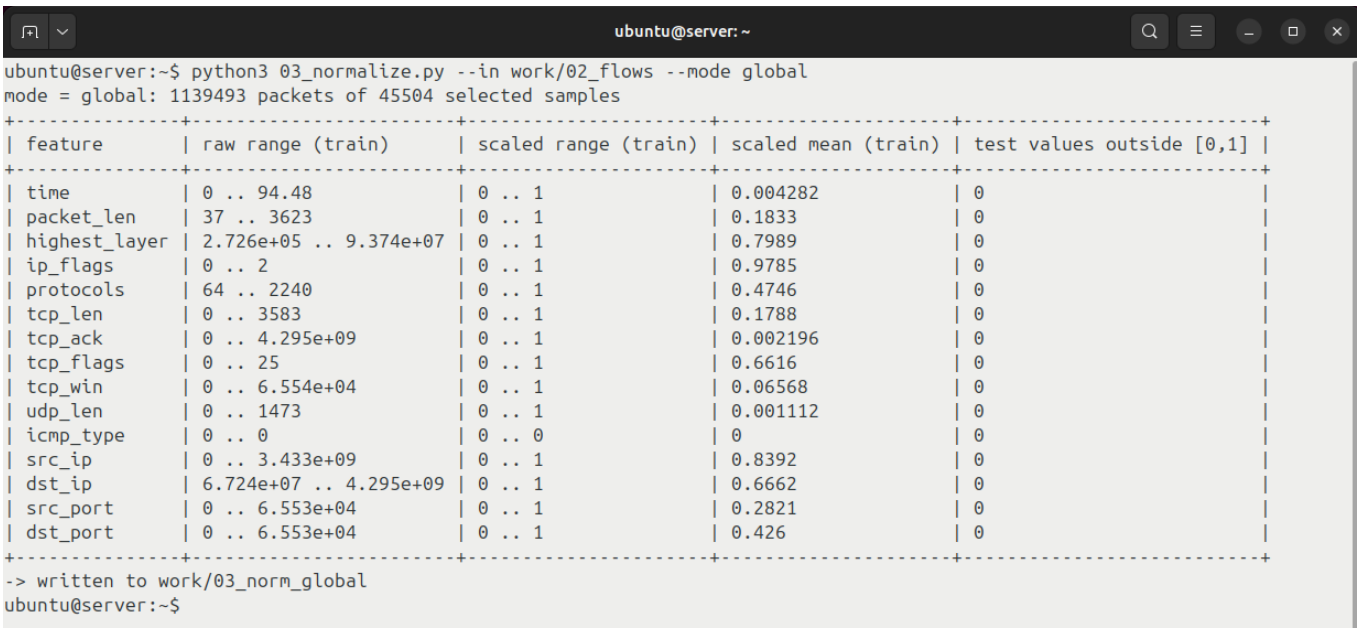


Figure 14: Global min-max normalization fitted on the training data

Compare the *raw range* column in Figure 14: `tcp_ack` goes up to $4.3 \cdot 10^9$, while `tcp_flags` is at most 25. After scaling, every feature lies in $[0, 1]$ on the training data. The last column counts test values that fall outside this range. Figure 15 shows the check of this step.

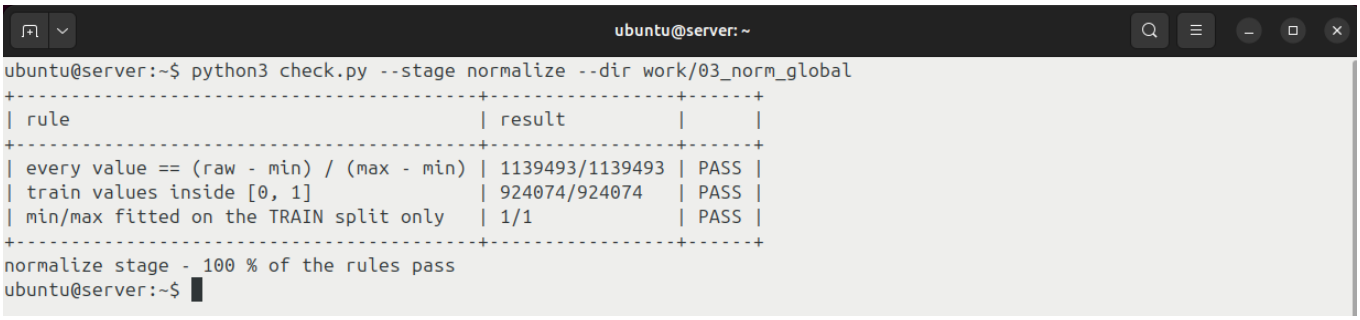


Figure 15: Checking the normalization step

Progress check: this step completes the `checkwork` goal `normalize_check` (Section 12).

6.4 Step 4: Feature Selection

```
python3 04_features.py --in work/03_norm_global --set lucid \
    --out work/04_feat_global_lucid
python3 check.py --stage features --dir work/04_feat_global_lucid
```

Figure 16 shows which features are kept, and Figure 17 the check of this step.



Figure 16: The 11 LUCID features are selected, identifiers are left out



Figure 17: Checking the feature step

Progress check: this step completes the `checkwork` goal `features_check` (Section 12).

6.5 Step 5: Shaping

```
python3 05_shape.py --in work/04_feat_global_lucid -n 100 \
    --out work/05_shape_global_lucid
python3 check.py --stage shape --dir work/05_shape_global_lucid
```

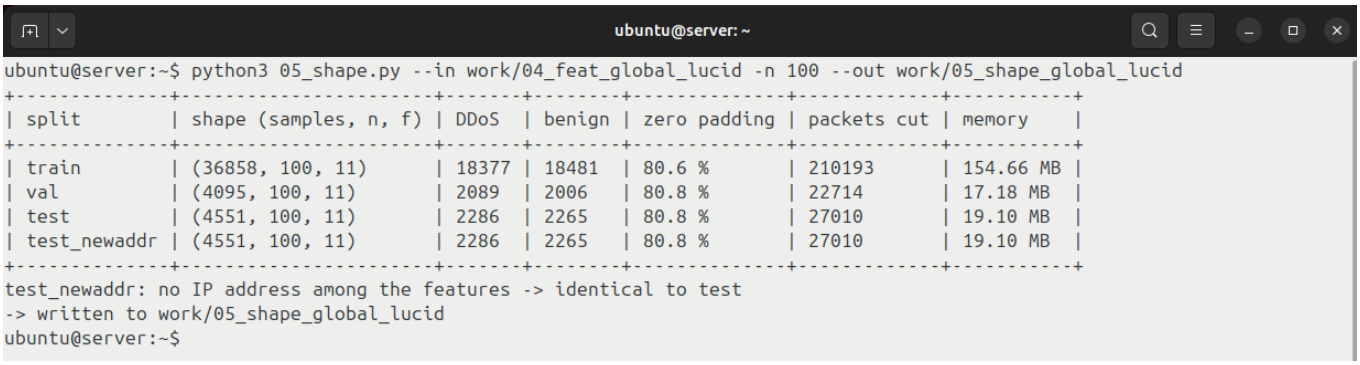


Figure 18: Every sample becomes a 100×11 matrix

Most flows are short (Figure 18): about 80 % of the rows are zero padding. The column *packets cut* counts the packets dropped from flows longer than 100 packets. Figure 19 shows the check of this step.



Figure 19: Checking the shape step

Progress check: this step completes the `checkwork` goal `shape_check` (Section 12).

7 Training, Inference and Evaluation

With the dataset prepared, you now train the LUCID CNN on the training split, use the saved model to classify the test split, and evaluate its predictions. These three steps form the basic cycle that every experiment in Section 8 repeats with different settings.

7.1 Step 6: Training the CNN

```
python3 06_train.py --data work/05_shape_global_lucid --model models/global.pt
```

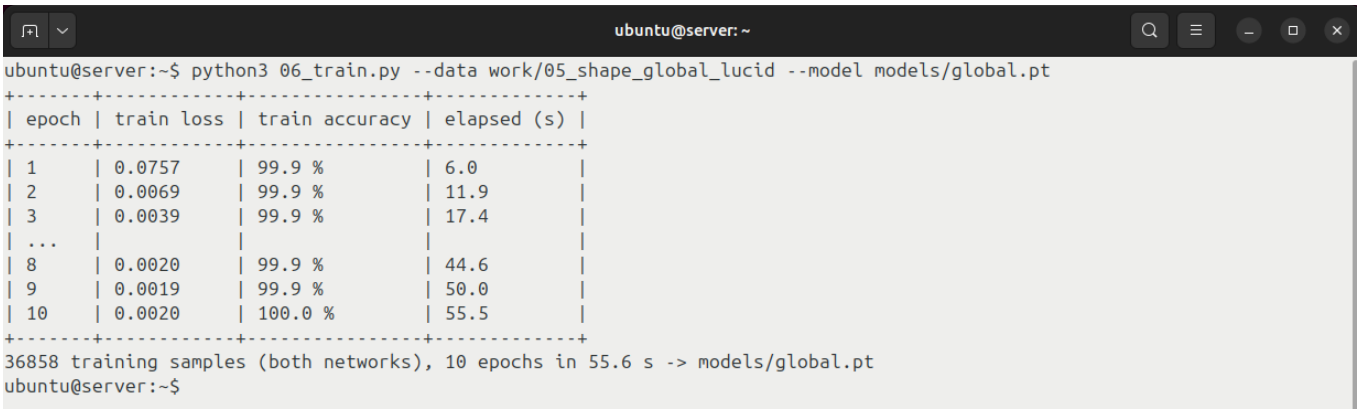


Figure 20: Training the LUCID CNN for 10 epochs

Watch the *train loss* column in Figure 20: it shows how quickly the model learns. You will compare it between the normalization modes later; the loss of every epoch is also saved next to the model (`models/global_train_log.json`) and drawn by `09_plot.py`. The training settings of Section 3.3 can be changed with `--epochs`, `--batch`, `--lr` and `--kernels`; keep the defaults for the lab, because the reference results are computed with them.

7.2 Step 7: Inference

The model file contains the weights *and* the settings of the pipeline it was trained with, so inference needs only the model and the data (Figure 21):

```
python3 07_infer.py --model models/global.pt --data work/05_shape_global_lucid \
    --split test --out results/global.csv
```

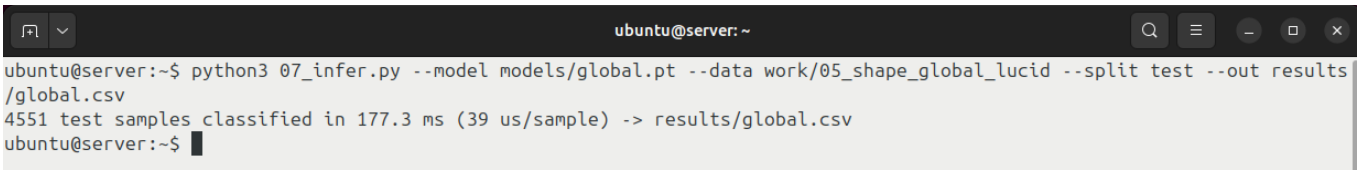


Figure 21: Classifying the 4,551 test samples

A sample is classified as DDoS when the model's probability is at least 0.5; this decision threshold can be changed with `--threshold`. The output file contains one line per sample with its true label, the probability of DDoS and the decision.

7.3 Step 8: Evaluation

```
python3 08_evaluate.py --pred results/global.csv --name global \
    --group normalization --model models/global.pt
```

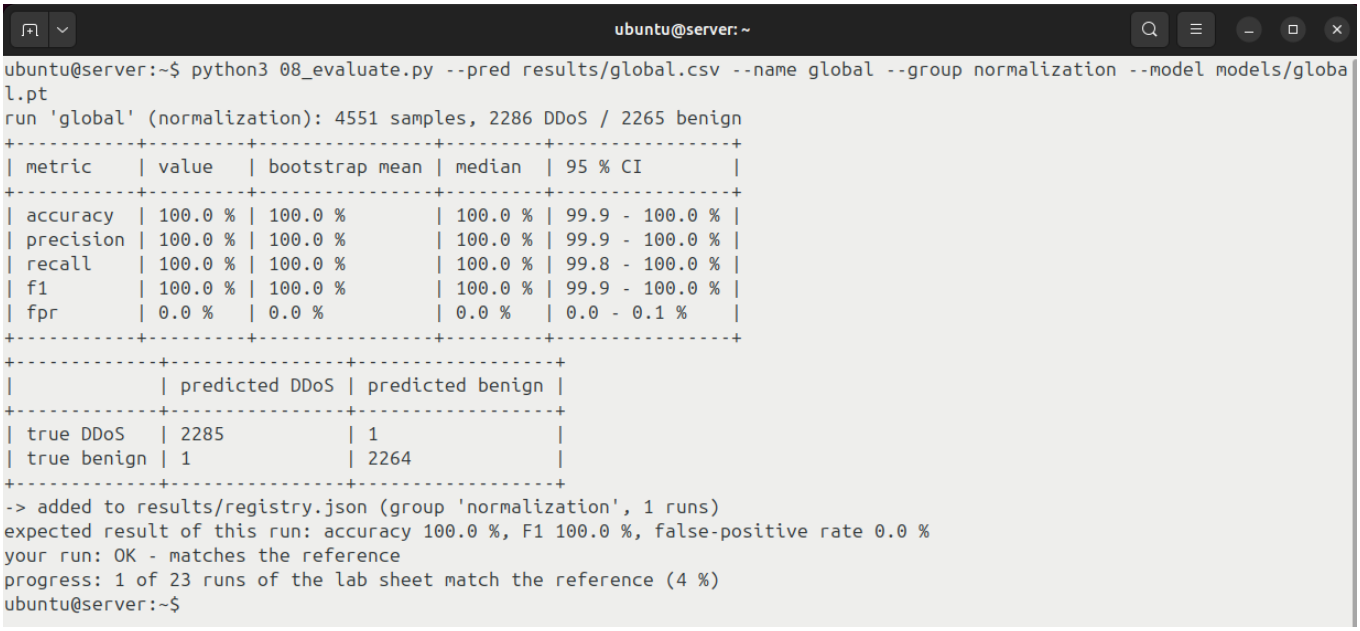


Figure 22: Metrics with 95 % bootstrap confidence intervals and the confusion matrix

The result (Figure 22) is also added to `results/registry.json`, which the plots read later. The last three lines tell you how well you did:

- **expected result of this run:** the result every student obtains with the commands of this manual (the lab is deterministic; two runs vary a little between computers and are checked more loosely, see Experiments 1 and 6);

- **your run:** `OK - matches the reference`, or `CHECK -` followed by what differs and which parameter to check, for example *n_packets is 10, expected 3: check the -n of 05_shape.py*;
- **progress:** how many of the 23 runs of this lab sheet you have completed correctly so far, as a number and a percentage (here 1 of 23, 4 %).

Every evaluation in the experiments ends with these lines, so after each run you see at once whether it is correct and how far you are.

8 Experiments

The following six experiments change one choice of the pipeline at a time and compare the results: the normalization mode, the feature set, the number of packets per sample, a classical threshold detector instead of the CNN, an explanation of what the trained models rely on, and finally the network the model is trained on. At the end, you check all runs against the reference results and draw one comparison figure per experiment.

8.1 Experiment 1: Normalization

Run the same pipeline for the other three modes. You can type the loop in one go:

```
for m in none theoretical batchnorm; do
  python3 03_normalize.py --in work/02_flows --mode $m
  python3 04_features.py --in work/03_norm_$m --set lucid \
      --out work/04_feat_${m}_lucid
  python3 05_shape.py --in work/04_feat_${m}_lucid -n 100 \
      --out work/05_shape_${m}_lucid
  python3 06_train.py --data work/05_shape_${m}_lucid --model models/${m}.pt
  python3 07_infer.py --model models/${m}.pt --data work/05_shape_${m}_lucid \
      --split test --out results/${m}.csv
  python3 08_evaluate.py --pred results/${m}.csv --name $m --group normalization \
      --model models/${m}.pt
done
```

This takes about three minutes. Together with the `global` run of Section 7 you should obtain the results of Table 3 on the test split:

Run (`--name`)	**Accuracy %**	**F1 %**	**FPR %**
global	100.0	100.0	0.0
none	97.7	97.6	0.0
theoretical	99.9	99.9	0.2
batchnorm	100.0	100.0	0.0

Table 3: Expected results of the normalization experiment

The three normalized modes reach almost the same accuracy; the clearest difference between the modes is in the training loss (Figure 36). Training on raw values is numerically unstable: tiny rounding differences between computers lead to a different model, so your result of `none` can differ from Table 3 (on another computer we measured 76.8 % instead of 97.7 %). For this run, `08_evaluate.py` and `check.py` only check that you used the right settings. Compare your `none` result with a classmate's: this instability is itself a result of the experiment.

8.2 Experiment 2: Feature Selection

This experiment uses the lab network (Section 2.2): the min-max scaling is fitted on its training samples (`--fit-on lab`), the models are trained on them (`--train-on lab`) and tested on its test samples (`--source lab`). First normalize once more, fitted on the lab network:

```
python3 03_normalize.py --in work/02_flows --mode global --fit-on lab \
    --out work/03_norm_global_lab
```

Then train one model per feature set and test every model twice: on the normal test split and on `test_newaddr`, where the attacker and benign addresses are swapped.

```
for s in lucid lucid_ids ids_only; do
  python3 04_features.py --in work/03_norm_global_lab --set $s \
      --out work/04_feat_lab_$s
  python3 05_shape.py --in work/04_feat_lab_$s -n 100 \
      --out work/05_shape_lab_$s
  python3 06_train.py --data work/05_shape_lab_$s --train-on lab \
      --model models/lab_$s.pt
  for sp in test test_newaddr; do
    python3 07_infer.py --model models/lab_$s.pt --data work/05_shape_lab_$s \
        --split $sp --source lab --out results/lab_${s}_$sp.csv
    python3 08_evaluate.py --pred results/lab_${s}_$sp.csv --name "$s ($sp)" \
        --group features --model models/lab_$s.pt
  done
done
```

Tip: Instead of a predefined set, `--features` takes any list of feature names, for example `--features time packet_len tcp_flags`, to try your own selection.

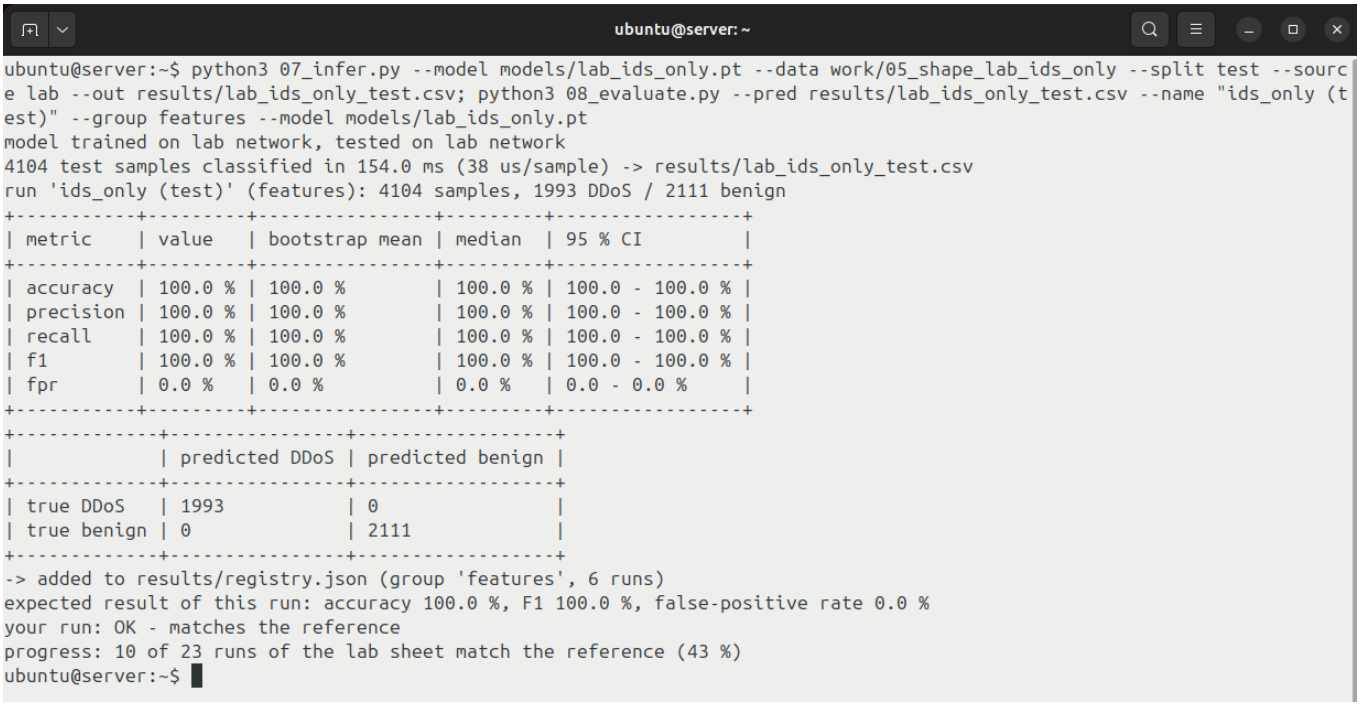


Figure 23: The address-only model on the normal test split

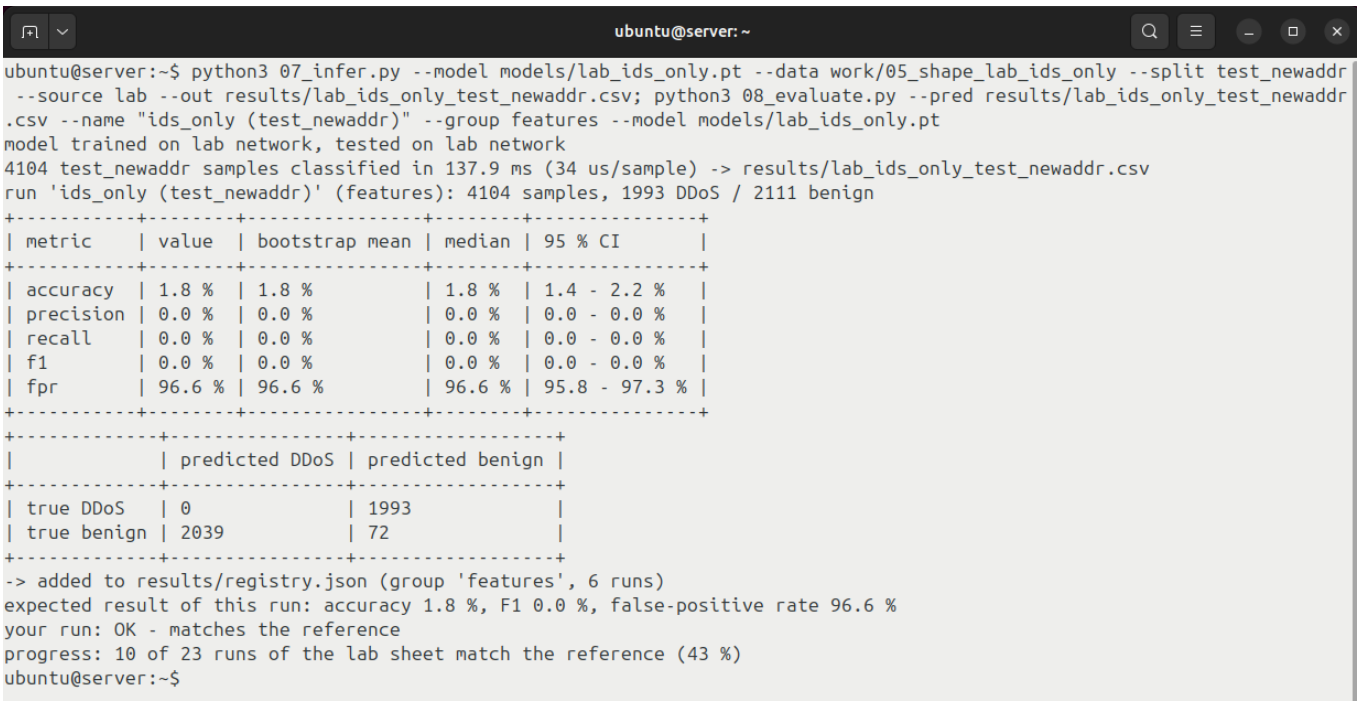


Figure 24: The same model after the addresses were swapped

Progress check: this step completes the `checkwork` goal `address_swap_tested` (Section 12).

On the normal test split the address-only model looks perfect (Figure 23). Once the addresses change, it is almost always wrong (Figure 24): it had learned *who* sends the traffic, not *what* the traffic looks like. Table 4 lists the expected results of all six runs (4,104 test samples of the lab network each).

Run (`--name`)	**Accuracy %**	**F1 %**	**FPR %**
lucid (test)	100.0	100.0	0.0
lucid (test_newaddr)	100.0	100.0	0.0
lucid_ids (test)	100.0	100.0	0.0
lucid_ids (test_newaddr)	40.4	5.3	24.6
ids_only (test)	100.0	100.0	0.0
ids_only (test_newaddr)	1.8	0.0	96.6

Table 4: Expected results of the feature selection experiment

8.3 Experiment 3: Shaping

Compare different numbers of packets per sample:

```
for n in 3 10 50 100; do
  python3 05_shape.py --in work/04_feat_global_lucid -n $n \
      --out work/05_shape_global_lucid_n$n
  python3 06_train.py --data work/05_shape_global_lucid_n$n \
      --model models/global_lucid_n$n.pt
  python3 07_infer.py --model models/global_lucid_n$n.pt \
      --data work/05_shape_global_lucid_n$n --split test \
      --out results/global_lucid_n$n.csv
```

```
    python3 08_evaluate.py --pred results/global_lucid_n$n.csv --name n=$n \
        --group shape \
        --model models/global_lucid_n$n.pt
done
```

Figure 25 shows the last evaluation of the loop, and Table 5 the expected results of all four runs.

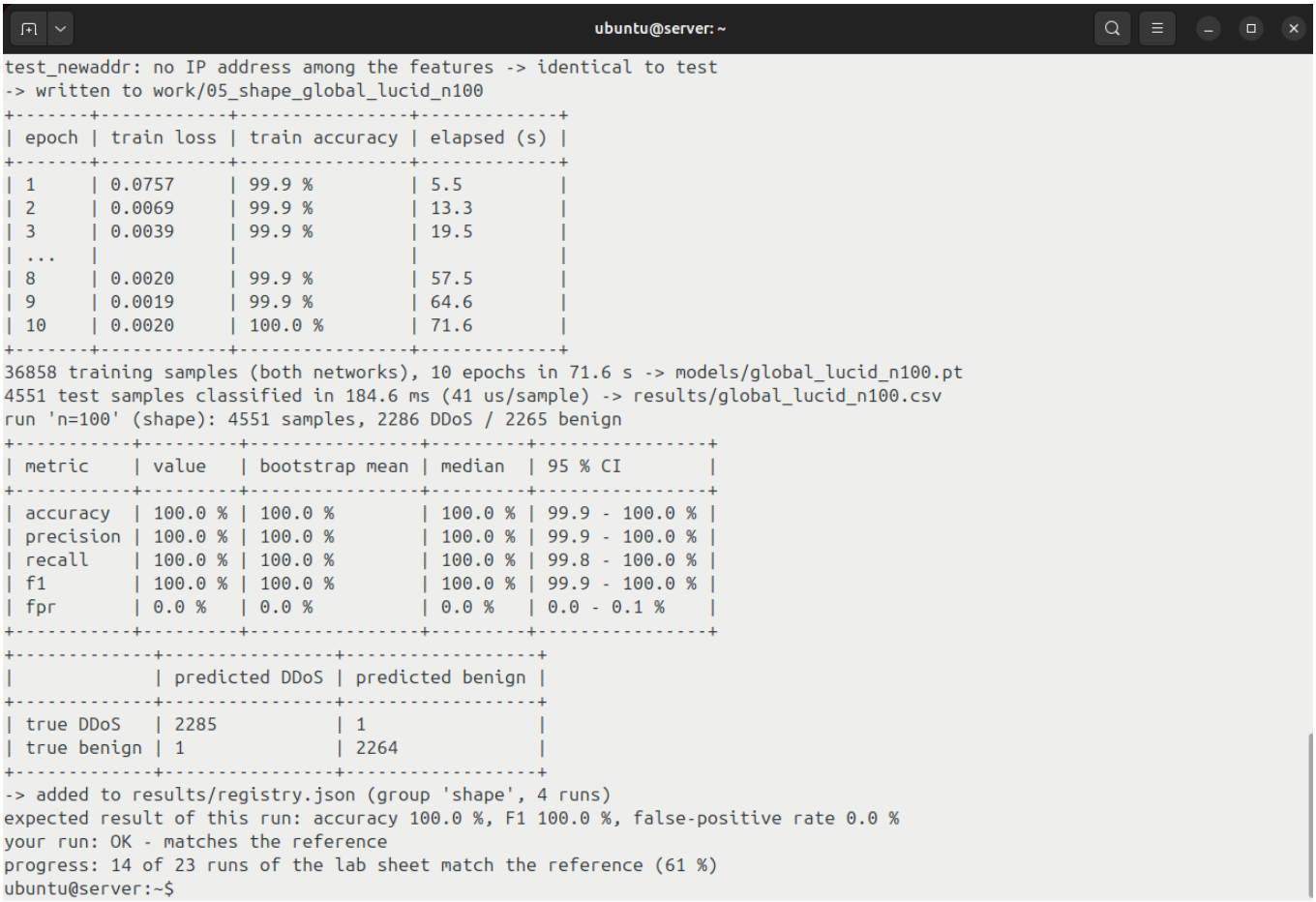


Figure 25: Evaluation of the shaping experiment ($n = 100$ shown last)

Run (`--name`)	**Accuracy %**	**F1 %**	**FPR %**
n=3	79.3	74.0	0.1
n=10	99.9	99.9	0.0
n=50	100.0	100.0	0.0
n=100	100.0	100.0	0.0

Table 5: Expected results of the shaping experiment

8.4 Experiment 4: Classical Detection vs. LUCID

`14_classical.py` is a classical threshold detector. In every window it counts the TCP SYN packets (connection attempts) that each source sends to the server per second, and flags every flow of a source above the threshold as DDoS. The threshold is chosen on the *training* split (the value that separates attacking and benign sources best) and then applied to the test split, just like a rule tuned by an administrator.

```
python3 14_classical.py --flows work/02_flows --rule syn --split test \
    --out results/classical_syn.csv
```

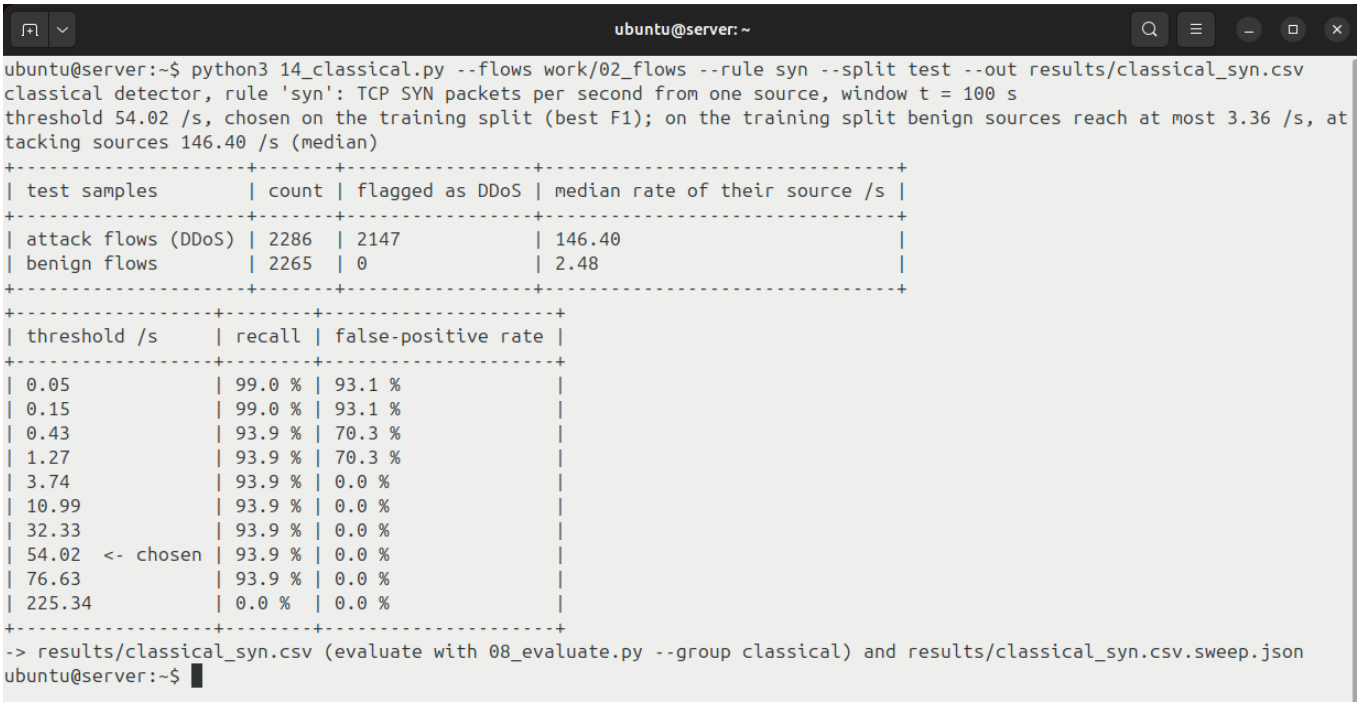


Figure 26: The classical SYN-rate rule on the test split

While the attack comes from one address, the rule catches every SYN flood and HTTP flood of the lab network without a single false alarm (Figure 26): benign sources send at most 3.4 SYN packets per second, the attacking sources about 146 (median). Its recall is still only 93.9 %: almost all flows it misses (131 of 139) are the UDP attacks of the CIC-DDoS2019 sample (DNS reflection and UDP-Lag). In those windows the attacker sends almost no SYN packets (about 0.3 per second), so a rule that counts SYN packets cannot see them. Now send the *same* attack flows from 50 bot addresses, as a botnet would:

```
python3 14_classical.py --flows work/02_flows --rule syn --split test --bots 50 \
    --out results/classical_syn_bots50.csv
```

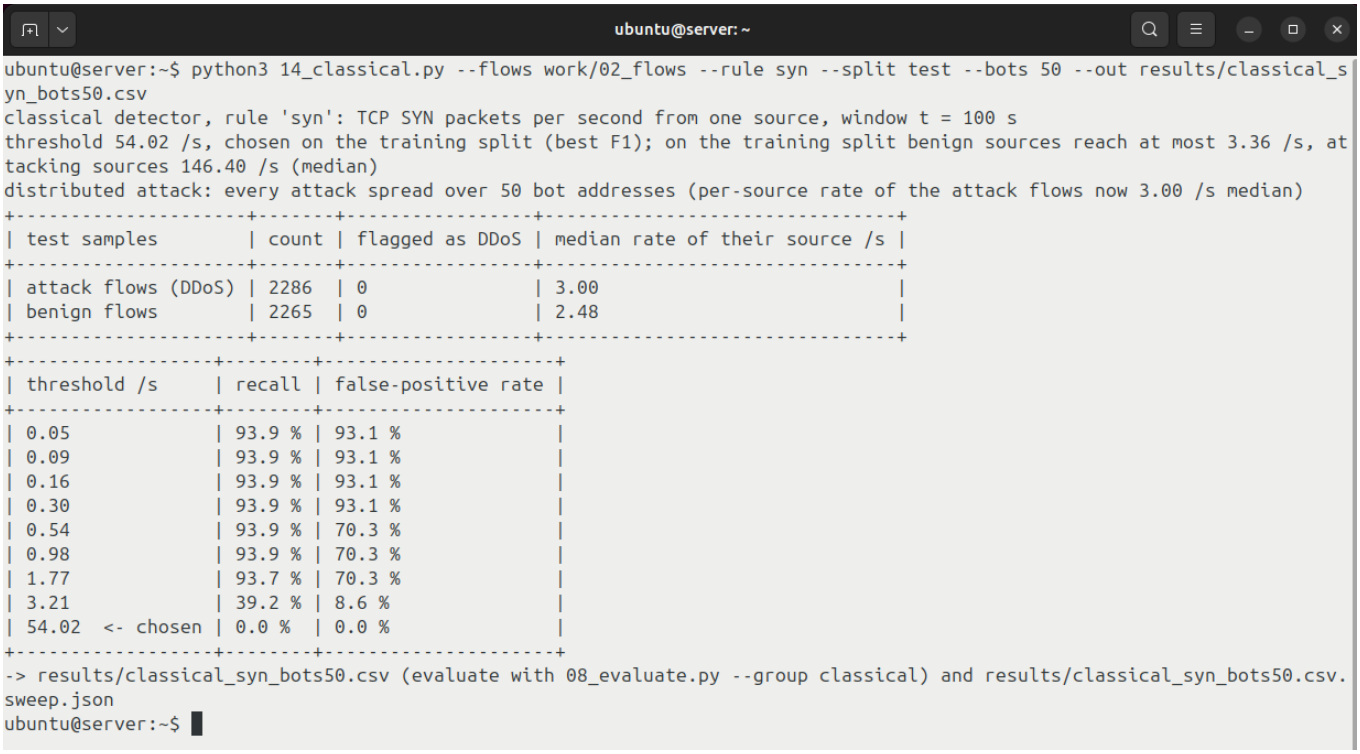


Figure 27: The same rule against the attack spread over 50 bots

Every bot now sends about 3 SYN packets per second (Figure 27), inside the range of the benign users. The tuned threshold catches nothing, and the sweep table shows that no other threshold works either: a threshold low enough to catch 39 % of the attack already flags about 9 % of the benign flows. LUCID judges every flow on its own and never sees addresses, so moving the attack flows to other addresses does not change any of its decisions.

Evaluate both rules and the LUCID model in the group `classical` (the LUCID predictions `results/global.csv` come from Step 7, Section 7.2). Run the three commands one after the other:

```
python3 08_evaluate.py --pred results/global.csv --name "LUCID CNN" \
    --group classical --model models/global.pt
python3 08_evaluate.py --pred results/classical_syn.csv --name "syn threshold" \
    --group classical
python3 08_evaluate.py --pred results/classical_syn_bots50.csv \
    --name "syn threshold (50 bots)" --group classical
```

Figure 28 shows the evaluation of the distributed case, and Table 6 the expected results of the three runs.

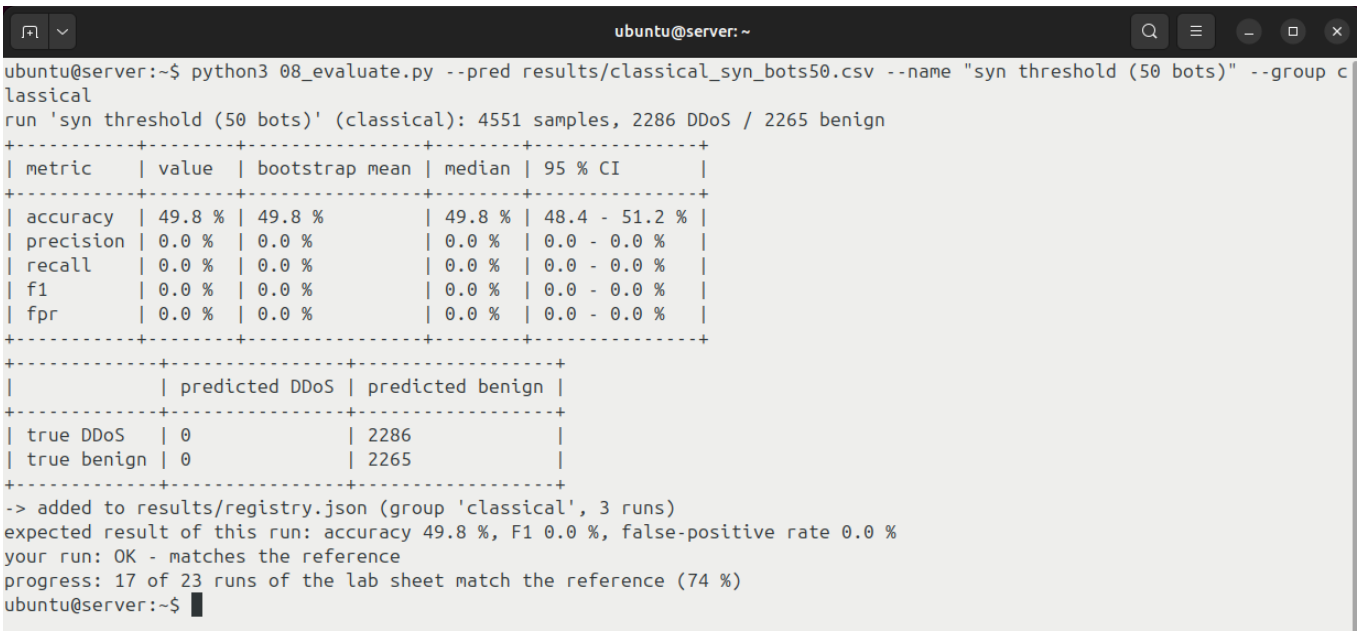


Figure 28: Evaluation of the classical rule against the distributed attack

Progress check: this step completes the `checkwork` goal `classical_tested` (Section 12).

Run (`--name`)	**Accuracy %**	**F1 %**	**FPR %**
LUCID CNN	100.0	100.0	0.0
syn threshold	96.9	96.9	0.0
syn threshold (50 bots)	49.8	0.0	0.0

Table 6: Expected results of the classical detection experiment

The accuracy of 49.8 % of the distributed case is not a half-working detector: it flags nothing, and half of the test samples happen to be benign.
Tip: `--rule pps` counts all packets per second instead of SYN packets; try it with and without `--bots`. `--threshold` sets a fixed threshold instead of the one chosen on the training split, so you can see how an administrator's guess performs.

8.5 Experiment 5: Explaining the Model's Decisions

A high accuracy does not tell *why* a model decides as it does. `15_explain.py` shuffles the values of one feature at a time between the packets of the test split and measures how much the accuracy drops (**permutation importance**). A feature whose shuffling hurts a lot is one the model relies on. Every feature is shuffled five times (`--repeats`), and the table shows the mean drop and its range. The script also shows the error rate by flow length. With `--split test_newaddr` the same analysis runs on the address-swapped test split.

```
python3 15_explain.py --model models/global.pt \
    --data work/05_shape_global_lucid
```

Figure 29 shows the output for the LUCID model, and Table 7 the three most important features of each model you explain in this experiment.

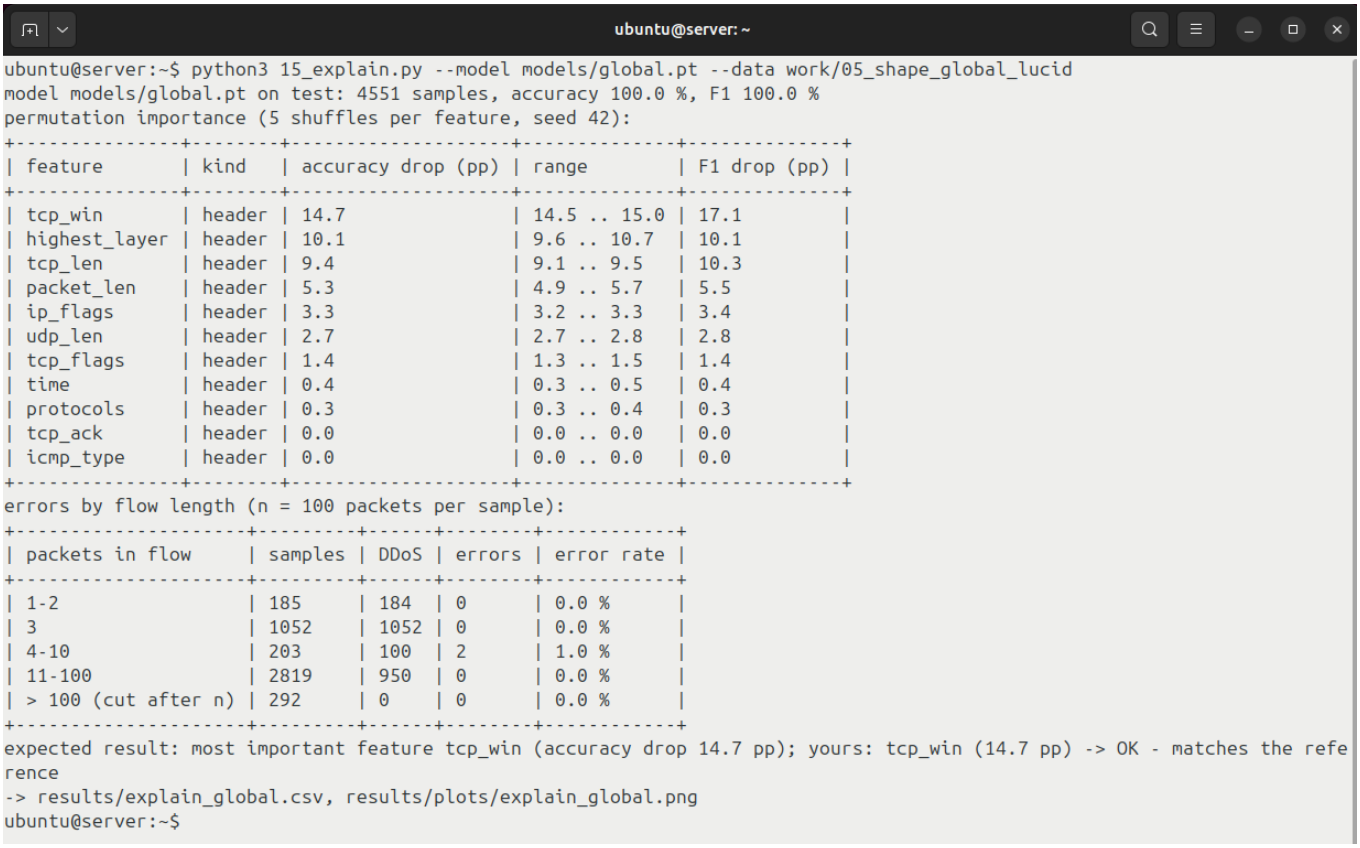


Figure 29: What the LUCID model relies on

Model	**1st feature**	**2nd feature**	**3rd feature**
LUCID model (`global`)	tcp_win 14.7	highest_layer 10.1	tcp_len 9.4
with addresses (`lab_lucid_ids`)	src_ip 37.6	all others 0.0	-
$n = 3$ (`global_lucid_n3`)	tcp_win 12.1	ip_flags 5.9	tcp_flags 1.7

Table 7: Expected permutation importance (drop in accuracy, percentage points)

The last line of the output compares your result with the expected one: the most important feature and its accuracy drop. The LUCID model relies on the TCP window size, the highest protocol layer and the payload length: properties of how the traffic behaves. Now explain the model that also saw the addresses (Experiment 2, explained on the lab network with `--source lab`) and the model with only $n = 3$ packets per sample (Experiment 3):

```
python3 15_explain.py --model models/lab_lucid_ids.pt \
    --data work/05_shape_lab_lucid_ids --source lab
python3 15_explain.py --model models/global_lucid_n3.pt \
    --data work/05_shape_global_lucid_n3
```

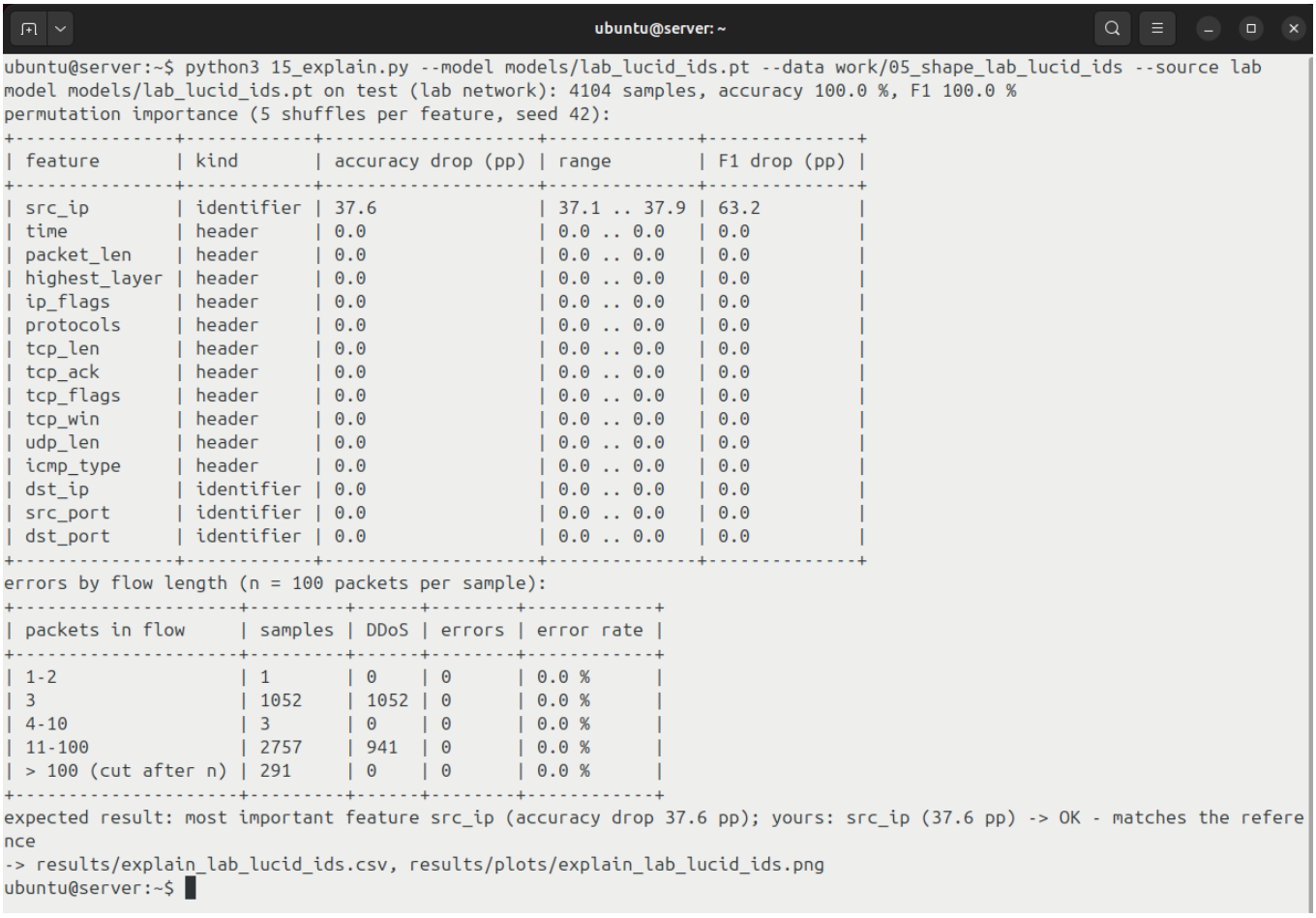


Figure 30: The model with addresses relies only on the source IP

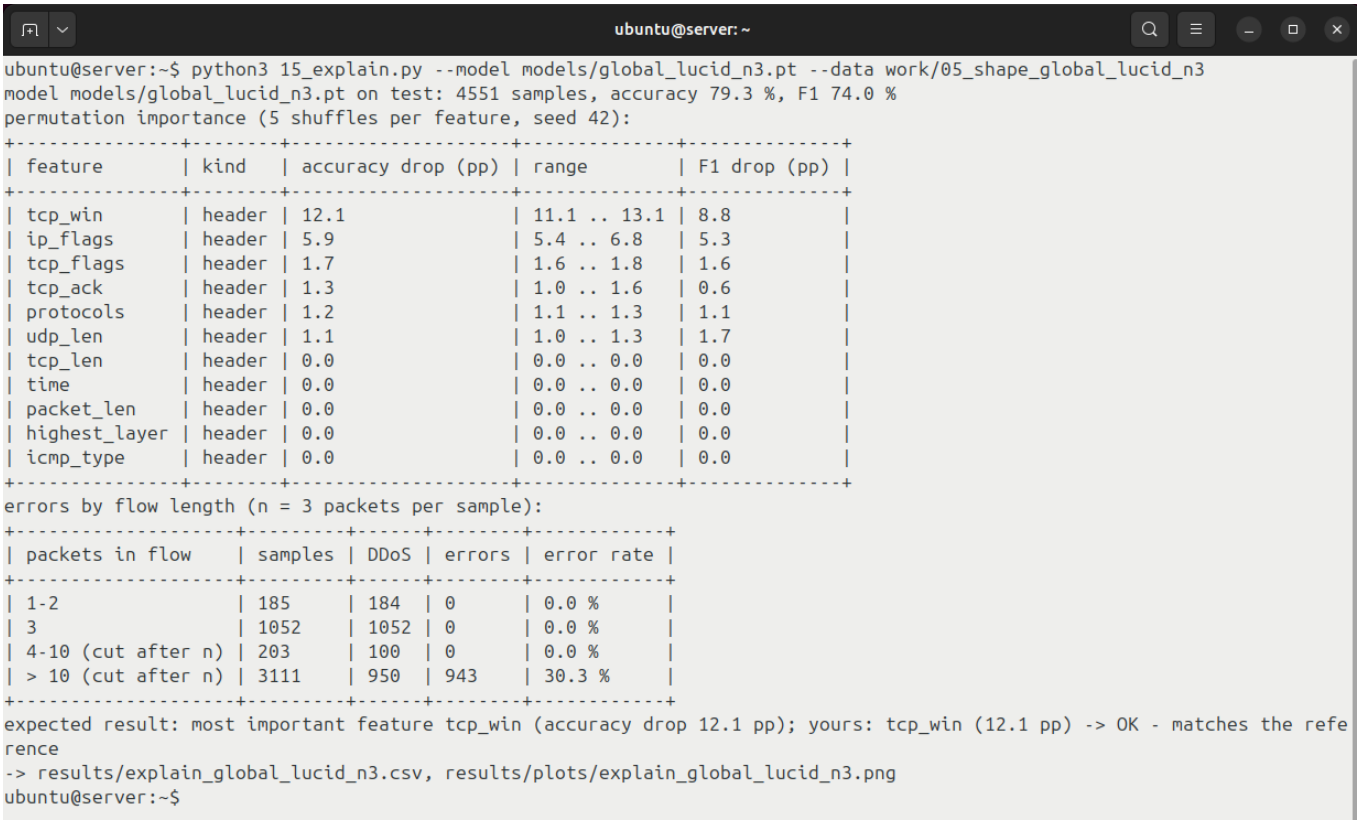


Figure 31: With $n = 3$, almost all long flows of the HTTP flood are missed

Progress check: this step completes the `checkwork` goal `decisions_explained` (Section 12). The model with addresses has learned only one thing: the source IP (Figure 30). Shuffling it costs 38 accuracy points, while no header feature matters at all; this is the shortcut that failed on `test_newaddr`. The $n = 3$ model misses 943 of the 950 attack flows longer than 10 packets (Figure 31): these are the HTTP-flood flows, whose first three packets (the TCP handshake) look exactly like a normal web request. The bar charts are saved in `results/plots/explain_*.png`; Figure 32 shows the one of the LUCID model.

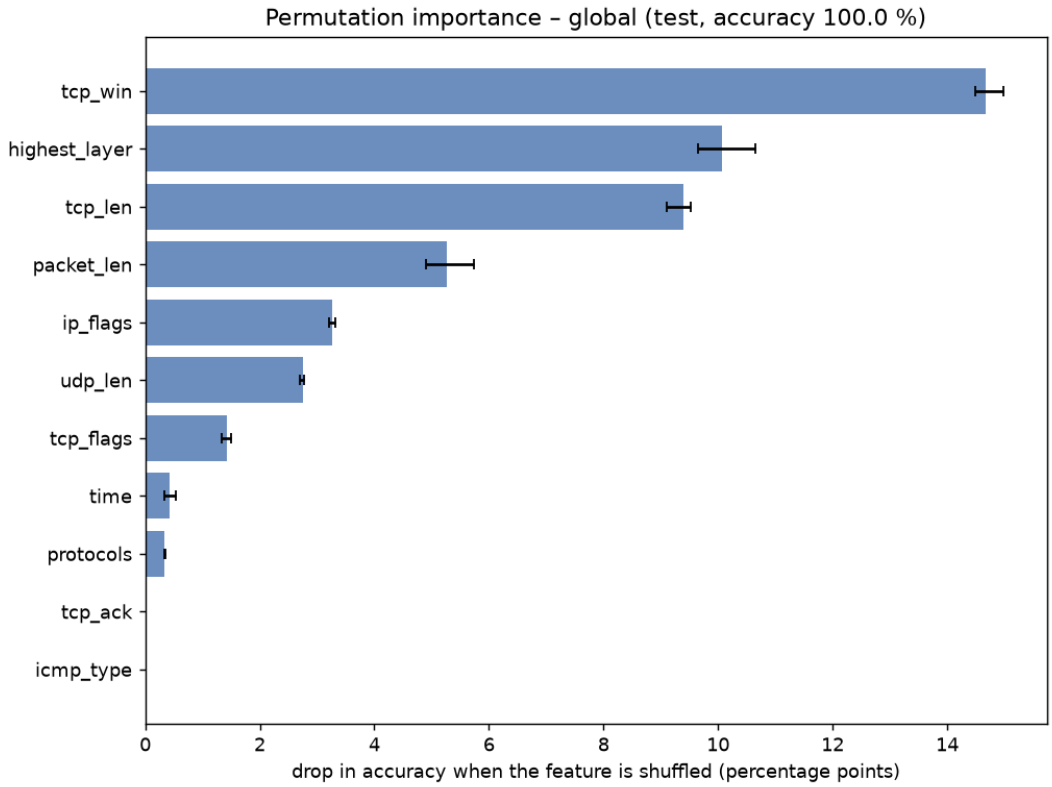


Figure 32: Permutation importance of the LUCID model

8.6 Experiment 6: Training on Another Network

Your model was trained on both networks of the dataset. A detector bought from a vendor, or trained on a public dataset, has never seen your network. To see what this means, train a model on the CIC-DDoS2019 sample only (min-max scaling fitted on it with `--fit-on cic`, trained with `--train-on cic`) and test it on each network separately with `--source`:

```
python3 03_normalize.py --in work/02_flows --mode global --fit-on cic \
    --out work/03_norm_global_cic
python3 04_features.py --in work/03_norm_global_cic --set lucid \
    --out work/04_feat_cic_lucid
python3 05_shape.py --in work/04_feat_cic_lucid -n 100 \
    --out work/05_shape_cic_lucid
python3 06_train.py --data work/05_shape_cic_lucid --train-on cic \
    --model models/cic_lucid.pt
```

Then test three models on both networks: the model trained on the CIC-DDoS2019 sample, the model trained on the lab network (`lucid` of Experiment 2) and your model of Step 6, trained on both:

```
for src in cic lab; do
  python3 07_infer.py --model models/cic_lucid.pt --data work/05_shape_cic_lucid \
      --split test --source $src --out results/cic_only_test_$src.csv
  python3 08_evaluate.py --pred results/cic_only_test_$src.csv \
      --name "cic only (test $src)" --group shift --model models/cic_lucid.pt
  python3 07_infer.py --model models/lab_lucid.pt --data work/05_shape_lab_lucid \
      --split test --source $src --out results/lab_only_test_$src.csv
```

```
    python3 08_evaluate.py --pred results/lab_only_test_$src.csv \
        --name "lab only (test $src)" --group shift --model models/lab_lucid.pt
    python3 07_infer.py --model models/global.pt --data work/05_shape_global_lucid \
        --split test --source $src --out results/both_test_$src.csv
    python3 08_evaluate.py --pred results/both_test_$src.csv \
        --name "cic + lab (test $src)" --group shift --model models/global.pt
done
```

`07_infer.py` names the network a model was trained on and the network it is tested on (Figure 33). Table 8 lists the expected results of the six runs.

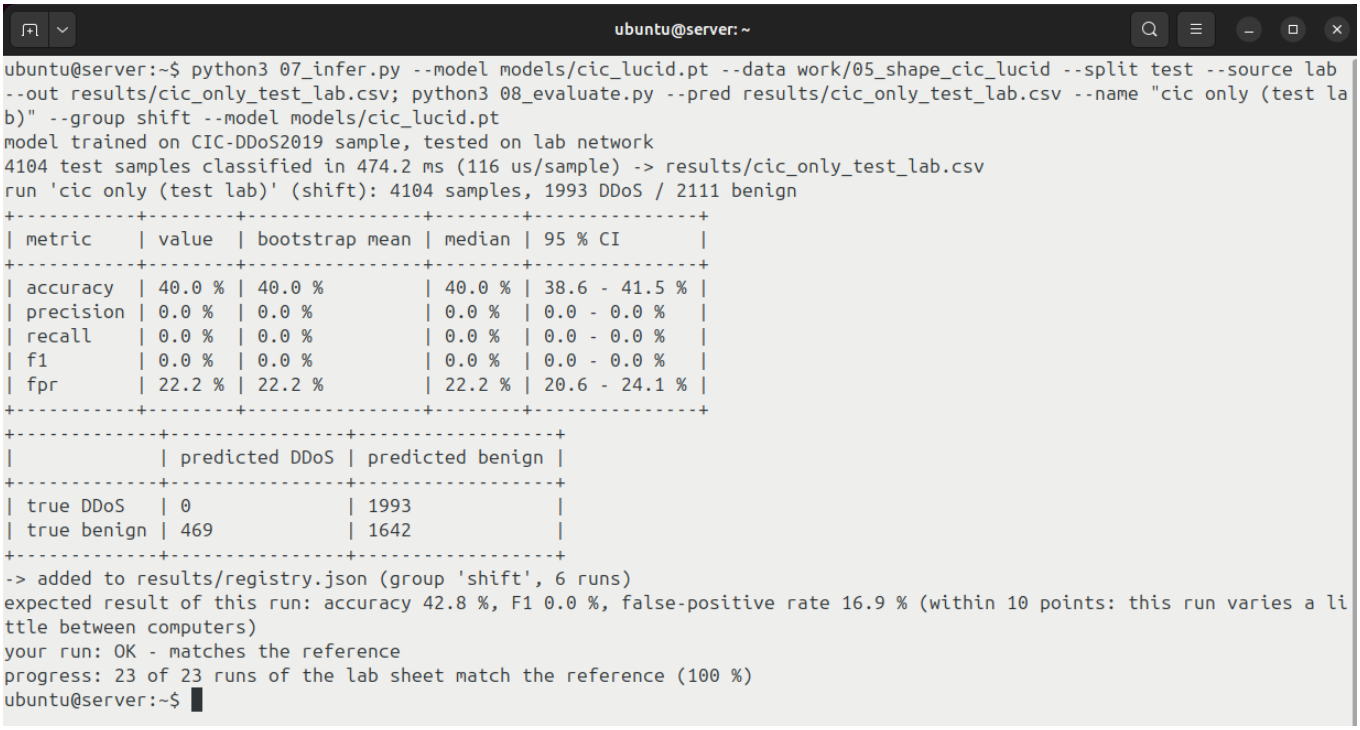


Figure 33: The model trained on the CIC-DDoS2019 sample, tested on the lab network

Progress check: this step completes the `checkwork` goal `network_shift_tested` (Section 12).

Run (`--name`)	**Accuracy %**	**F1 %**	**FPR %**
cic only (test cic)	99.8	99.8	0.0
lab only (test cic)	54.6	49.4	5.8
cic + lab (test cic)	99.6	99.7	0.6
cic only (test lab)	42.8	0.0	16.9
lab only (test lab)	100.0	100.0	0.0
cic + lab (test lab)	100.0	100.0	0.0

Table 8: Expected results of training on another network (447 test samples of the CIC-DDoS2019 sample, 4,104 of the lab network)

Each model is almost perfect on the network it was trained on. The model trained on the CIC-DDoS2019 sample does not find a single attack of the lab network (F1 0 %) and even flags benign users there; the model trained on the lab network misses two thirds of the CIC-DDoS2019 attacks,

whose attack types (DNS reflection, UDP-Lag) it has never seen. Only the model trained on both networks detects the attacks of both. This is why the lab trains on the LUCID data *and* on a recording of the network it defends in the post-lab.
Note: The model trained on the CIC-DDoS2019 sample sees values of the lab network that lie far outside its training range. Its result on the lab network therefore differs a little between computers (we measured an accuracy of 40.0–43.8 % and a false-positive rate of 14.8–22.2 % on four machines); `08_evaluate.py` and `check.py` accept it within 10 points. Its recall is always 0 %.

8.7 Checking Your Results Against the Reference

The lab is deterministic: with the same commands, every student obtains the results in the tables above (except for the two runs that vary between computers, Experiments 1 and 6). `check.py` can therefore check all your runs at once. For each of the 23 runs of this lab sheet it compares the settings your model was trained with (normalization mode, feature set, number of packets n, the network it was fitted and trained on, or the classical rule), the number of test samples and the metrics (within 0.5 percentage points) with the reference. Figure 34 shows the output for a complete and correct lab:

```
python3 check.py --stage results --dir results
```

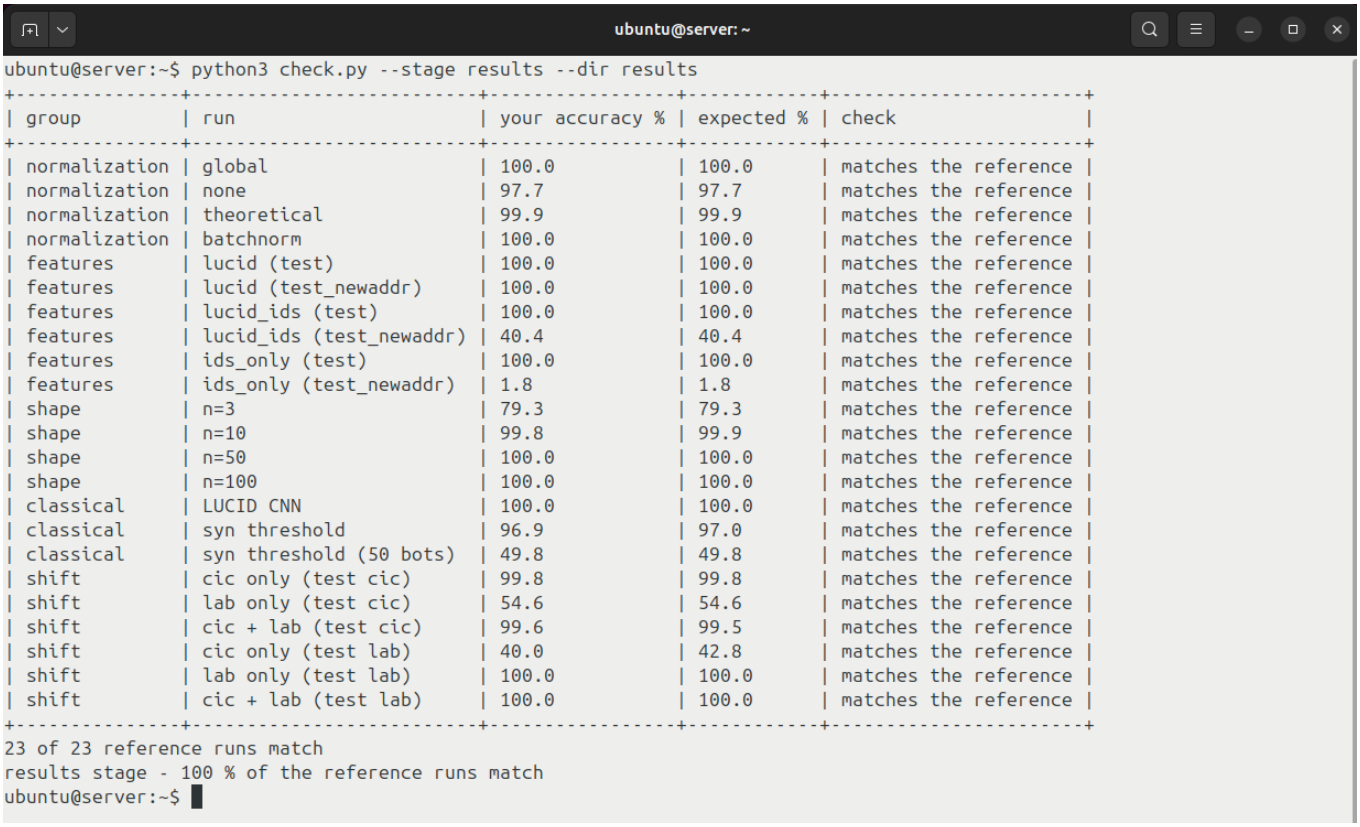


Figure 34: All runs of the lab sheet match the reference results

Progress check: this step completes the `checkwork` goal `results_check` (Section 12).
If a run does not match, the last column says what to check, for example:

```
| shape | n=3 | 99.9 | 79.3 | n_packets is 10, expected 3: check the -n of 05_shape.py |
```

A run that is missing is listed as *not run yet*. Repeat the step with the right parameters (with the same `--name` and `--group`, the new result replaces the old one) and run the check again until all 23 runs match.

8.8 Comparing the Experiments

Draw one figure per experiment. The same tables are printed on the screen (Figure 35):

```
python3 09_plot.py --groups normalization features shape classical shift
```

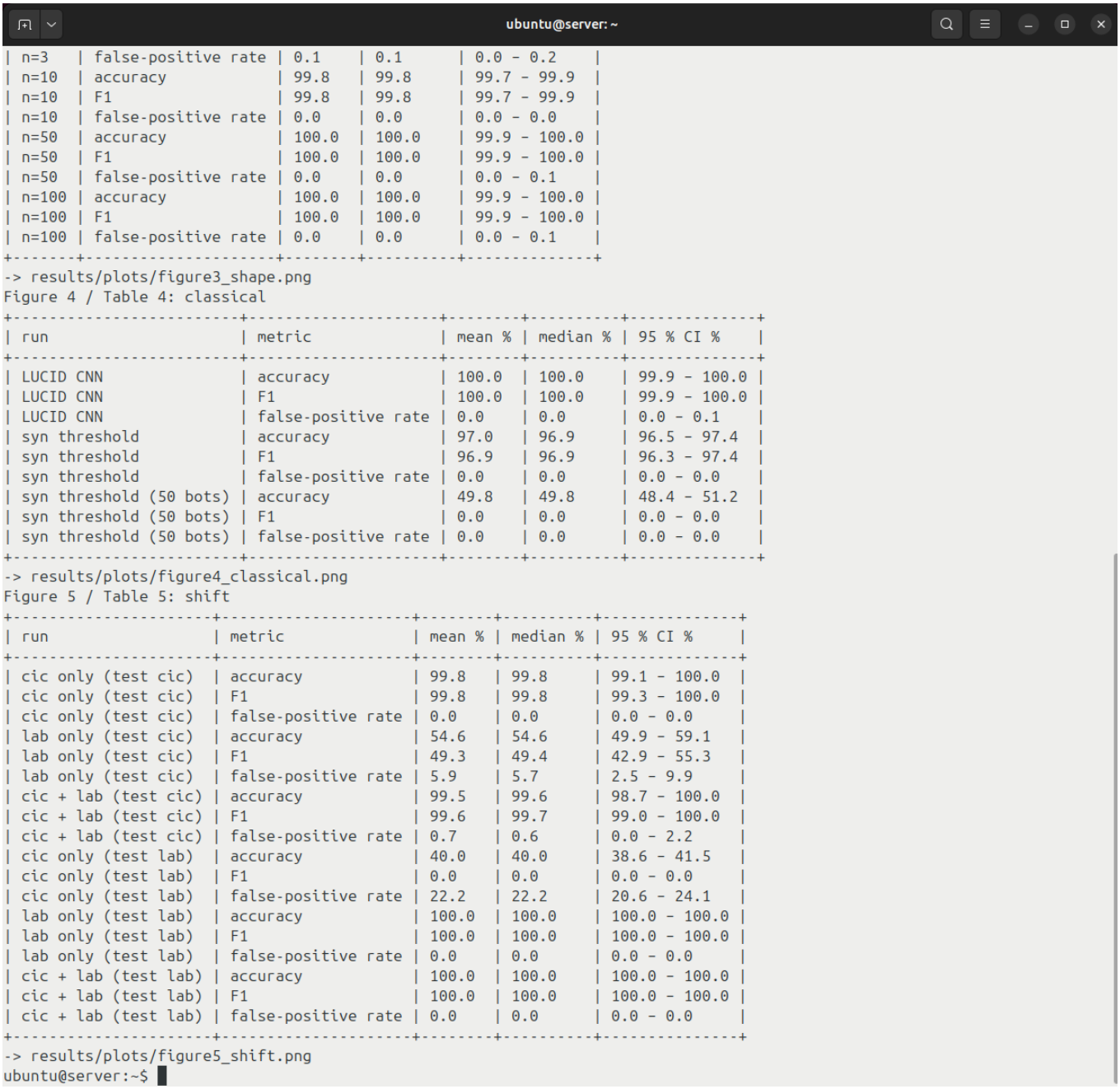


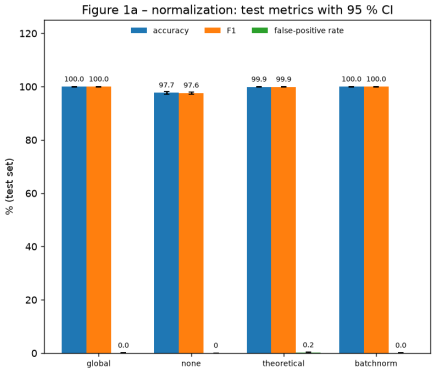
Figure 35: Comparison tables printed by `09_plot.py`

Open a figure with `feh` (the option `-.` fits the figure to the window; press `q` to close it):

```
feh -. results/plots/figure1_normalization.png
```

The five figures are shown in Figure 36 (normalization), Figure 37 (feature selection), Figure 38 (shaping), Figure 39 (classical detection) and Figure 40 (training on another network). In each of them, the left panel shows the metrics with their 95 % confidence intervals, the middle panel the training loss per epoch (for the classical rules: the recall and false-positive rate of every threshold),

and the right panel the mean, median and confidence interval of every metric. The panel titles inside the images (e.g. “Figure 1a”) are the numbers `09_plot.py` prints on the screen.



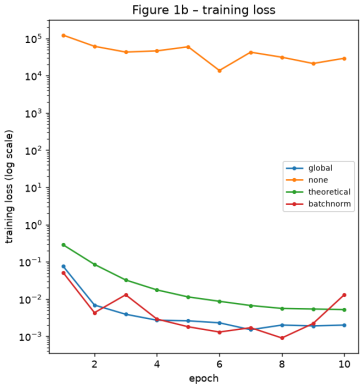
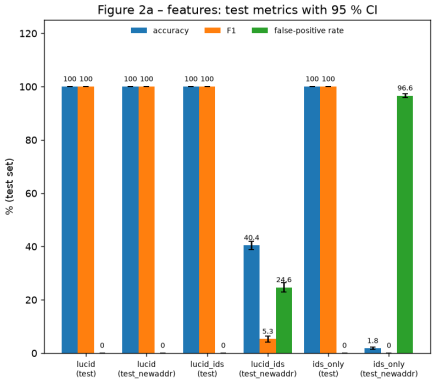


Table 1 – bootstrap statistics (4551 test samples per run)

run	metric	mean %	median %	95 % CI %
global	accuracy	100.0	100.0	99.9 – 100.0
global	F1	100.0	100.0	99.9 – 100.0
global	false-positive rate	0.0	0.0	0.0 – 0.1
none	accuracy	97.7	97.7	97.2 – 98.1
none	F1	97.6	97.7	97.1 – 98.1
none	false-positive rate	0.0	0.0	0.0 – 0.0
theoretical	accuracy	99.9	99.9	99.8 – 100.0
theoretical	F1	99.9	99.9	99.8 – 100.0
theoretical	false-positive rate	0.2	0.2	0.0 – 0.4
batchnorm	accuracy	100.0	100.0	99.9 – 100.0
batchnorm	F1	100.0	100.0	99.9 – 100.0
batchnorm	false-positive rate	0.0	0.0	0.0 – 0.1

Figure 36: Normalization experiment: metrics with confidence intervals (left), training loss (middle), table (right)



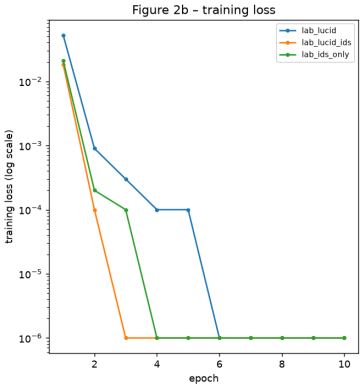
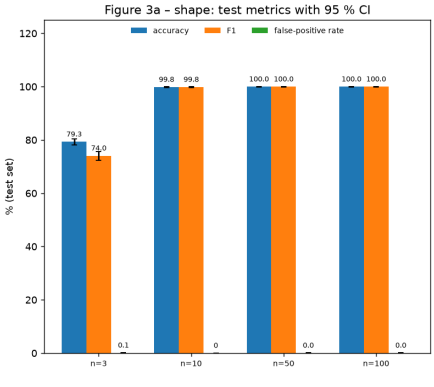


Table 2 – bootstrap statistics (4104 test samples per run)

run	metric	mean %	median %	95 % CI %
lucid (test)	accuracy	100.0	100.0	100.0 – 100.0
lucid (test)	F1	100.0	100.0	100.0 – 100.0
lucid (test)	false-positive rate	0.0	0.0	0.0 – 0.0
lucid (test_newaddr)	accuracy	100.0	100.0	100.0 – 100.0
lucid (test_newaddr)	F1	100.0	100.0	100.0 – 100.0
lucid (test_newaddr)	false-positive rate	0.0	0.0	0.0 – 0.0
lucid_ids (test)	accuracy	100.0	100.0	100.0 – 100.0
lucid_ids (test)	F1	100.0	100.0	100.0 – 100.0
lucid_ids (test)	false-positive rate	0.0	0.0	0.0 – 0.0
lucid_ids (test_newaddr)	accuracy	40.4	40.4	38.9 – 41.9
lucid_ids (test_newaddr)	F1	5.3	5.3	4.2 – 6.5
lucid_ids (test_newaddr)	false-positive rate	24.7	24.7	22.9 – 26.4
ids_only (test)	accuracy	100.0	100.0	100.0 – 100.0
ids_only (test)	F1	100.0	100.0	100.0 – 100.0
ids_only (test)	false-positive rate	0.0	0.0	0.0 – 0.0
ids_only (test_newaddr)	accuracy	1.8	1.8	1.4 – 2.2
ids_only (test_newaddr)	F1	0.0	0.0	0.0 – 0.0
ids_only (test_newaddr)	false-positive rate	96.6	96.6	95.8 – 97.3

Figure 37: Feature selection experiment: normal test split vs. swapped addresses



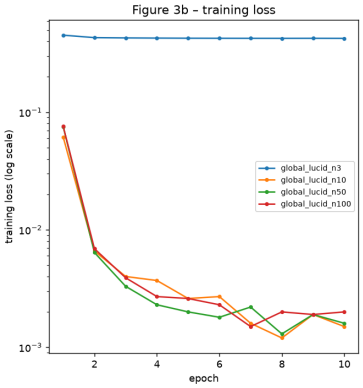
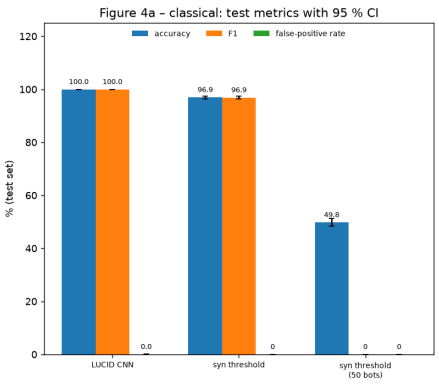


Table 3 – bootstrap statistics (4551 test samples per run)

run	metric	mean %	median %	95 % CI %
n=3	accuracy	79.3	79.3	78.1 – 80.4
n=3	F1	74.0	74.0	72.3 – 75.6
n=3	false-positive rate	0.1	0.1	0.0 – 0.2
n=10	accuracy	99.8	99.8	99.7 – 99.9
n=10	F1	99.8	99.8	99.7 – 99.9
n=10	false-positive rate	0.0	0.0	0.0 – 0.0
n=50	accuracy	100.0	100.0	99.9 – 100.0
n=50	F1	100.0	100.0	99.9 – 100.0
n=50	false-positive rate	0.0	0.0	0.0 – 0.1
n=100	accuracy	100.0	100.0	99.9 – 100.0
n=100	F1	100.0	100.0	99.9 – 100.0
n=100	false-positive rate	0.0	0.0	0.0 – 0.1

Figure 38: Shaping experiment: number of packets per sample



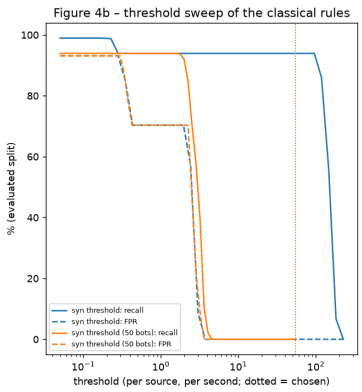
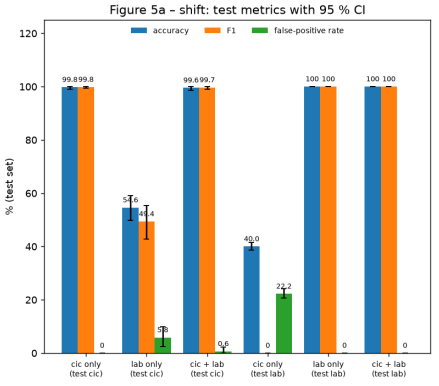


Table 4 – bootstrap statistics (4551 test samples per run)

run	metric	mean %	median %	95 % CI %
LUCID CNN	accuracy	100.0	100.0	99.9 – 100.0
LUCID CNN	F1	100.0	100.0	99.9 – 100.0
LUCID CNN	false-positive rate	0.0	0.0	0.0 – 0.1
syn threshold	accuracy	97.0	96.9	96.5 – 97.4
syn threshold	F1	96.9	96.9	96.3 – 97.4
syn threshold	false-positive rate	0.0	0.0	0.0 – 0.0
syn threshold (50 bots)	accuracy	49.8	49.8	48.4 – 51.2
syn threshold (50 bots)	F1	0.0	0.0	0.0 – 0.0
syn threshold (50 bots)	false-positive rate	0.0	0.0	0.0 – 0.0

Figure 39: Classical detection vs. LUCID: metrics (left), recall and false-positive rate of every threshold (middle), table (right)



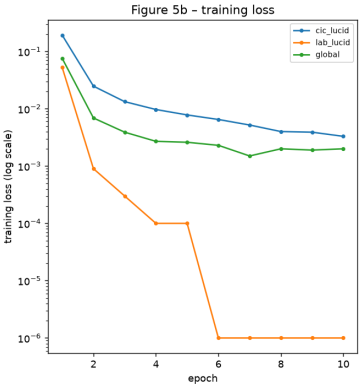


Table 5 – bootstrap statistics (447 or 4104 test samples per run)

run	metric	mean %	median %	95 % CI %
cic only (test cic)	accuracy	99.8	99.8	99.1 – 100.0
cic only (test cic)	F1	99.8	99.8	99.3 – 100.0
cic only (test cic)	false-positive rate	0.0	0.0	0.0 – 0.0
lab only (test cic)	accuracy	54.6	54.6	49.9 – 59.1
lab only (test cic)	F1	49.3	49.4	42.9 – 55.3
lab only (test cic)	false-positive rate	5.9	5.7	2.5 – 9.9
cic + lab (test cic)	accuracy	99.5	99.6	98.7 – 100.0
cic + lab (test cic)	F1	99.6	99.7	99.0 – 100.0
cic + lab (test cic)	false-positive rate	0.7	0.6	0.0 – 2.2
cic only (test lab)	accuracy	40.0	40.0	38.6 – 41.5
cic only (test lab)	F1	0.0	0.0	0.0 – 0.0
cic only (test lab)	false-positive rate	22.2	22.2	20.6 – 24.1
lab only (test lab)	accuracy	100.0	100.0	100.0 – 100.0
lab only (test lab)	F1	100.0	100.0	100.0 – 100.0
lab only (test lab)	false-positive rate	0.0	0.0	0.0 – 0.0
cic + lab (test lab)	accuracy	100.0	100.0	100.0 – 100.0
cic + lab (test lab)	F1	100.0	100.0	100.0 – 100.0
cic + lab (test lab)	false-positive rate	0.0	0.0	0.0 – 0.0

Figure 40: Training on another network: each model tested on the CIC-DDoS2019 sample and on the lab network

Progress check: this step completes the `checkwork` goals `normalization_compared`, `features_compared`, `shape_compared`, `classical_compared`, `shift_compared` (Section 12).
Questions for your analysis:

- The normalized modes reach almost the same test accuracy. Look at the training loss instead: which mode learns fastest and most stably, and why does the raw (`none`) loss stay so large? Why can the `none` result differ between two computers when the others do not?
- Why does `lucid` keep its accuracy on `test_newaddr` while `lucid_ids` drops to about 40 % and `ids_only` to almost zero?
- Why is $n = 3$ clearly worse than $n = 10$? Which packets of a SYN flood or an HTTP flood does the model miss when only three are kept?
- Do the confidence intervals of two runs overlap? What does that mean for your conclusions?
- In the middle panel of Figure 39, is there any threshold that catches the distributed attack without flagging benign users? What would an attacker have to change to evade LUCID instead?
- Which features does the LUCID model rely on (Experiment 5), and why do they differ between a SYN flood and normal web traffic?

- Why does the model trained on the CIC-DDoS2019 sample fail on the lab network (Experiment 6)? What would you need before you deploy a detector trained on a public dataset in your own network?

8.9 Collecting Your Results

Every script prints its result on the screen *and* saves it in a file, so you can come back to it later and use it as evidence in your analysis and in the post-incident report. After each script, look at its result before you go on: read the table on the screen, compare it with the expected result, and note the numbers you will need. Table 9 lists where every script saves its result.

Script	**Saved result**	**Content**
`01_window.py` – `05_shape.py`	`work/01_window/` ... `work/05_shape_*/`	the prepared data of every step, with its `pipeline.json`
`06_train.py`	`models/<name>.pt`, `models/<name>_train_log.json`	the trained model and its loss per epoch
`07_infer.py`	`results/<name>.csv`	one line per test sample: true label, probability of DDoS, decision
`08_evaluate.py`	`results/registry.json`	the metrics and confidence intervals of every evaluated run
`09_plot.py`	`results/plots/figure1_normalization.png` ... `figure5_shift.png`	the comparison figures with their tables
`14_classical.py`	`results/classical_*.csv` (and `.sweep.json`)	the decisions of the classical rule and its threshold sweep
`15_explain.py`	`results/explain_<model>.csv`, `results/plots/explain_<model>.png`	the permutation importance of a model
`12_build_replay.py`	`replay/`	the replay files of the post-lab and `manifest.txt`
`10_detect_block.py`	`results/incident_live.csv`	every flow of the live test with its decision and block time
`11_incident_report.py`	`results/incident_report.txt`	the post-incident report

Table 9: Where the scripts save their results

To see what you have collected so far, list the result folders (Figure 41):

```
ls results results/plots
```

```
ubuntu@server:~$ ls results results/plots
results:
batchnorm.csv                        explain_lab_lucid_ids.csv      lab_lucid_ids_test_newaddr.csv
both_test_cic.csv                    global.csv                     lab_lucid_test.csv
both_test_lab.csv                    global_lucid_n10.csv           lab_lucid_test_newaddr.csv
cic_only_test_cic.csv                global_lucid_n100.csv          lab_only_test_cic.csv
cic_only_test_lab.csv                global_lucid_n3.csv            lab_only_test_lab.csv
classical_syn.csv                    global_lucid_n50.csv           none.csv
classical_syn.csv.sweep.json         incident_live.csv              plots
classical_syn_bots50.csv             incident_report.txt            registry.json
classical_syn_bots50.csv.sweep.json  lab_ids_only_test.csv          theoretical.csv
explain_global.csv                   lab_ids_only_test_newaddr.csv
explain_global_lucid_n3.csv          lab_lucid_ids_test.csv

results/plots:
explain_global.png           explain_lab_lucid_ids.png  figure2_features.png  figure4_classical.png
explain_global_lucid_n3.png  figure1_normalization.png  figure3_shape.png     figure5_shift.png
ubuntu@server:~$
```

Figure 41: The results collected after the experiments

A result file can be looked at directly in the terminal, for example the first predictions of a model (Figure 42):

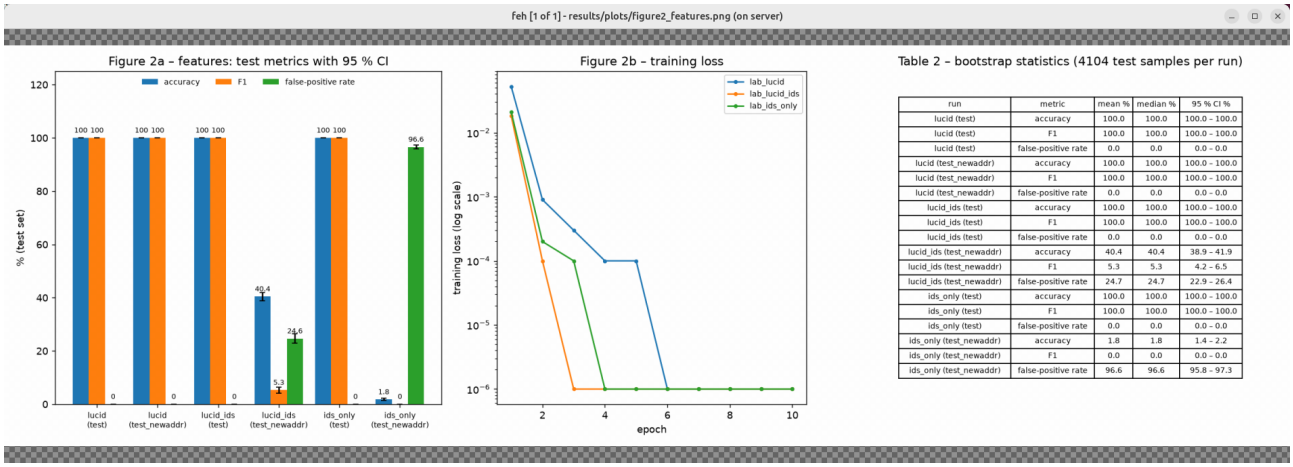
```
head -5 results/global.csv
```

```
ubuntu@server:~$ head -5 results/global.csv
sample,label,prob_ddos,pred
1,0,0.006207,0
3,0,0.989584,1
5,0,0.006251,0
9,0,0.000000,0
ubuntu@server:~$
```

Figure 42: The first lines of a prediction file

Figures open in their own window with `feh` (Figure 43); press `q` to close the window:

```
feh -. results/plots/figure2_features.png
```



run	metric	mean %	median %	95 % CI %
lucid (test)	accuracy	100.0	100.0	100.0 – 100.0
lucid (test)	F1	100.0	100.0	100.0 – 100.0
lucid (test)	false-positive rate	0.0	0.0	0.0 – 0.0
lucid (test_newaddr)	accuracy	100.0	100.0	100.0 – 100.0
lucid (test_newaddr)	F1	100.0	100.0	100.0 – 100.0
lucid (test_newaddr)	false-positive rate	0.0	0.0	0.0 – 0.0
lucid_ids (test)	accuracy	100.0	100.0	100.0 – 100.0
lucid_ids (test)	F1	100.0	100.0	100.0 – 100.0
lucid_ids (test)	false-positive rate	0.0	0.0	0.0 – 0.0
lucid_ids (test_newaddr)	accuracy	40.4	40.4	38.9 – 41.9
lucid_ids (test_newaddr)	F1	5.3	5.3	4.2 – 6.5
lucid_ids (test_newaddr)	false-positive rate	24.7	24.7	22.9 – 26.4
ids_only (test)	accuracy	100.0	100.0	100.0 – 100.0
ids_only (test)	F1	100.0	100.0	100.0 – 100.0
ids_only (test)	false-positive rate	0.0	0.0	0.0 – 0.0
ids_only (test_newaddr)	accuracy	1.8	1.8	1.4 – 2.2
ids_only (test_newaddr)	F1	0.0	0.0	0.0 – 0.0
ids_only (test_newaddr)	false-positive rate	96.6	96.6	95.8 – 97.3

Figure 43: A comparison figure opened with feh

You do not have to copy these files yourself: when you stop the lab, Labtainer collects the whole home directory of the server, including `results/` and `models/`, into the archive that is submitted for grading (Section 11).

9 Post-Lab: Live Detection and Incident Response

In this phase you defend the server with the model you trained in Step 6, `models/global.pt` (trained on both networks; on the lab network it reaches 100 %, Table 8).

Why the validation split? A validation split is normally used to tune training, for example to choose the number of epochs or the best model. In this lab these settings are already known: they come from the LUCID paper and from tuning the lab to run on one virtual CPU in Labtainer. We therefore do not need the validation split for model selection. Instead, it is used to **simulate a real attack**: its flows were never used for training, and they are replayed from two attacker containers and one benign client against the server. This shows how the model you trained yourself behaves under a real attack and whether it can still classify the traffic correctly. This is how the lab is adapted to an industry scenario. At the end, you write a **post-incident report** as a security professional would (Section 10).

9.1 Step 0: SSH Access to the Traffic Containers

The attack is started from the server over SSH. Like an incident response team preparing its tools, create a key and copy it to the three traffic containers (the password is `ubuntu`):

```
ssh-keygen -t ed25519
ssh-copy-id ubuntu@attacker1
ssh-copy-id ubuntu@attacker2
ssh-copy-id ubuntu@client
```

Press Enter at the questions of `ssh-keygen` to accept the defaults, and answer `yes` when `ssh-copy-id` asks whether to trust a host. Figure 44 shows the end of the key setup.

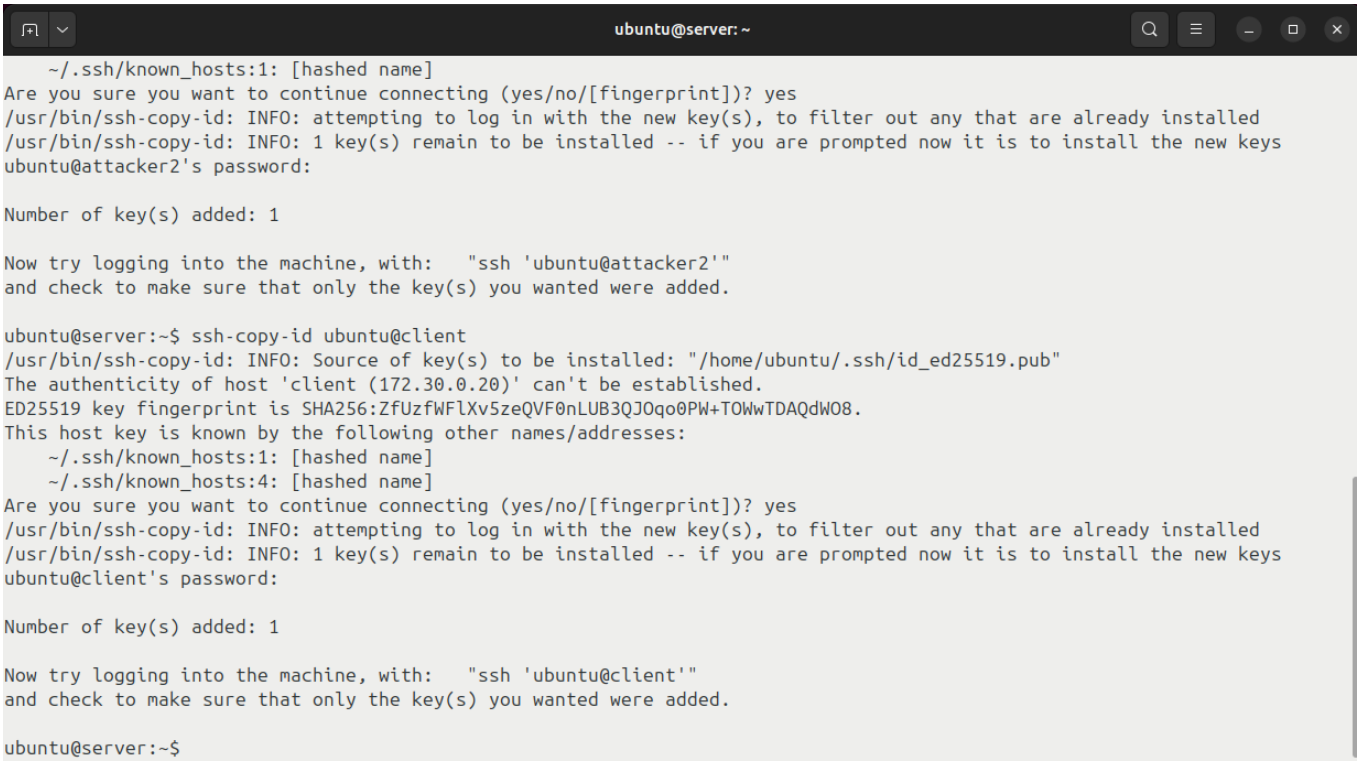


Figure 44: Setting up key-based SSH access

9.2 Step 1: Building the Replay Files

```
python3 12_build_replay.py --flows work/02_flows --victim 172.30.0.10 --out replay
```

The validation flows of the lab network (those of the victim `172.30.0.10`) are split into two attack files and one benign file (Figure 45). They come from the whole recording of the lab network (about 20 minutes); their start times are spread over 90 seconds (`--duration`), so the whole incident fits into one detection window, while the packets inside every flow keep their recorded timing. Flows that would merge into one when replayed from a single address (the same ports in two recorded windows) are kept only once; the line *left out as duplicates* counts them.

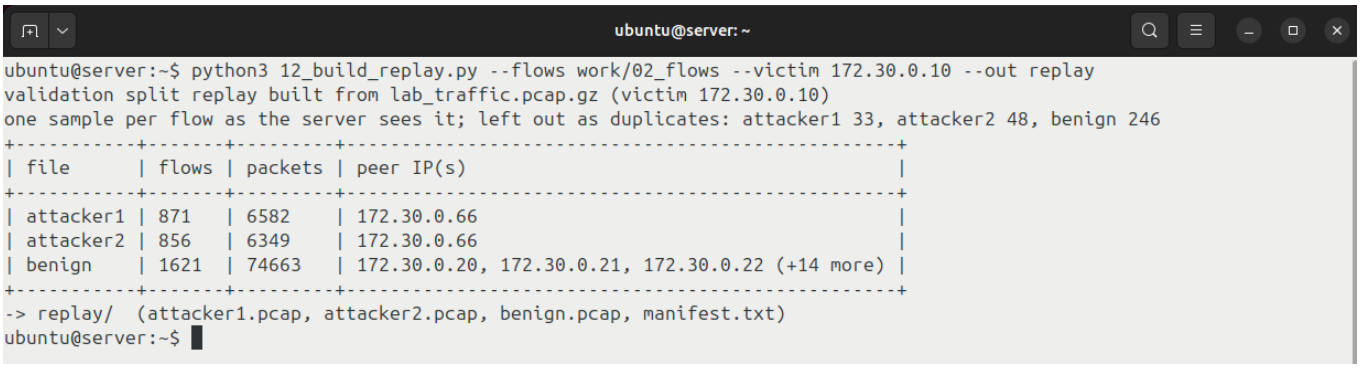


Figure 45: The validation flows split into two attack files and one benign file

9.3 Step 2: Starting the Detector (Terminal 1)

```
sudo python3 10_detect_block.py --model models/global.pt --iface eth0 \
    --victim 172.30.0.10 --block --seconds 150 --out results/incident_live.csv
```

The detector watches the server's interface, applies exactly the same pipeline as in training, and re-classifies the open window every 5 seconds. Every flow classified as DDoS appears as an `ALERT` line; after 3 alerts from the same address it is blocked with iptables (`BLOCK` line). The detector waits until traffic to the server arrives and then runs for 150 seconds from the first packet, so you can start the next step whenever you are ready.

Three settings decide when a flow is judged and when an address is blocked:

- `--interval 5`: the open window is classified again every 5 seconds, so attackers are blocked while the attack is still going on, not only when the 100-second window closes.
- `--idle 2`: before the window closes, only flows without a packet for 2 seconds (finished flows) are judged; a connection that is still running would otherwise be judged on its first packets only.
- `--min-alerts 3`: an address is blocked only after three of its flows were flagged, so a single false alarm does not cut off a legitimate user.

The decision threshold can be changed with `--threshold` (0.5 by default). The detector can also analyse a recorded file without touching the firewall, for example to look at one attacker's traffic first:

```
python3 10_detect_block.py --model models/global.pt --pcap replay/attacker1.pcap \
    --victim 172.30.0.10 --out results/incident_offline.csv
```

9.4 Step 3: Launching the Attack (Terminal 2)

Switch to the second server terminal and start the replay:

```
./13_launch_replay.sh
```

The script copies the files to `attacker1`, `attacker2` and `client` and starts them together. Every flow sent is shown as a `SEND` line: Figure 46 shows the start of the replay and Figure 47 its end. Before the replay starts, the script prepares the incident:

- it removes the firewall blocks a previous run left for the three containers, so the incident can be repeated (the lines *removed the block of ...* in Figure 46 appear only when you run it again);
- it maps the recorded addresses to the container that replays them, so the server sees the traffic coming from `attacker1`, `attacker2` and `client`;
- it keeps the server from answering the replayed packets (the recordings already contain the server's answers), so the flows look exactly as recorded.

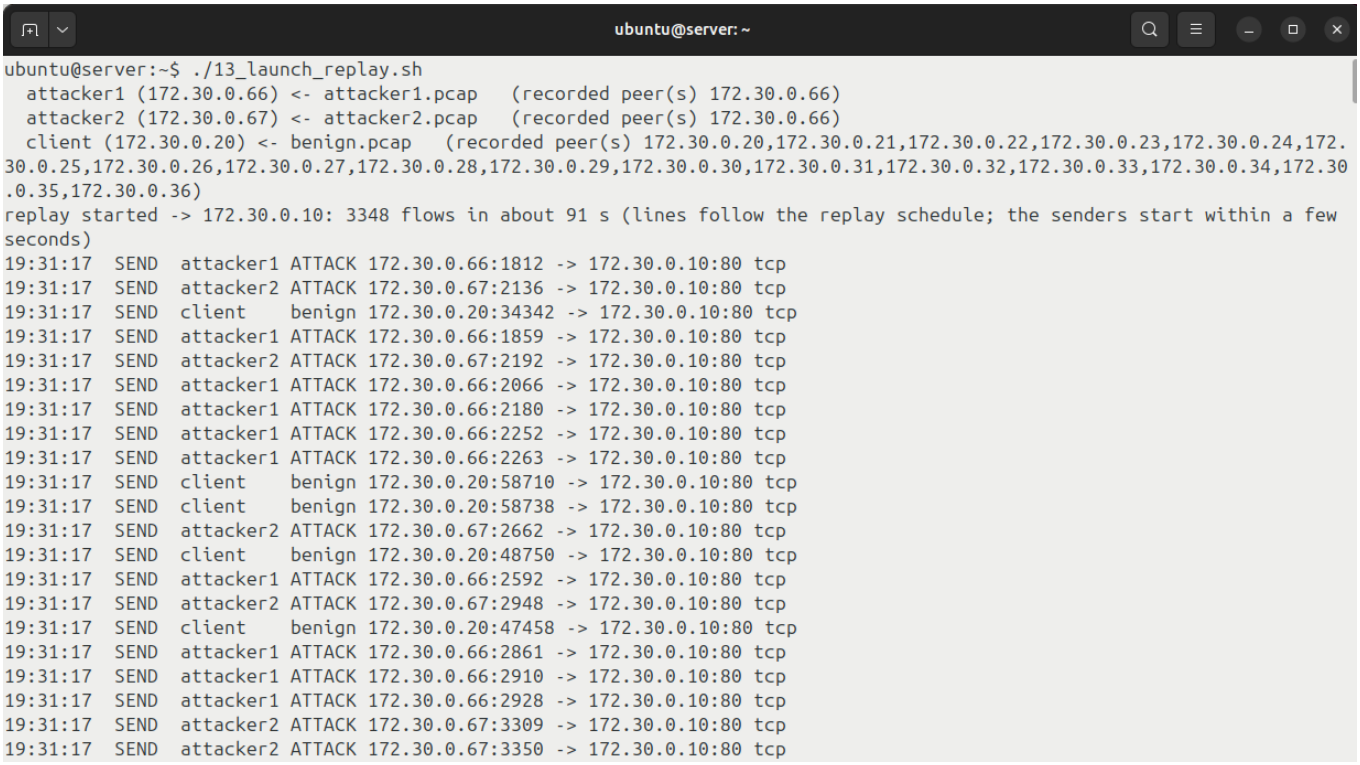


Figure 46: Terminal 2: the files are mapped to the three containers and the replay starts

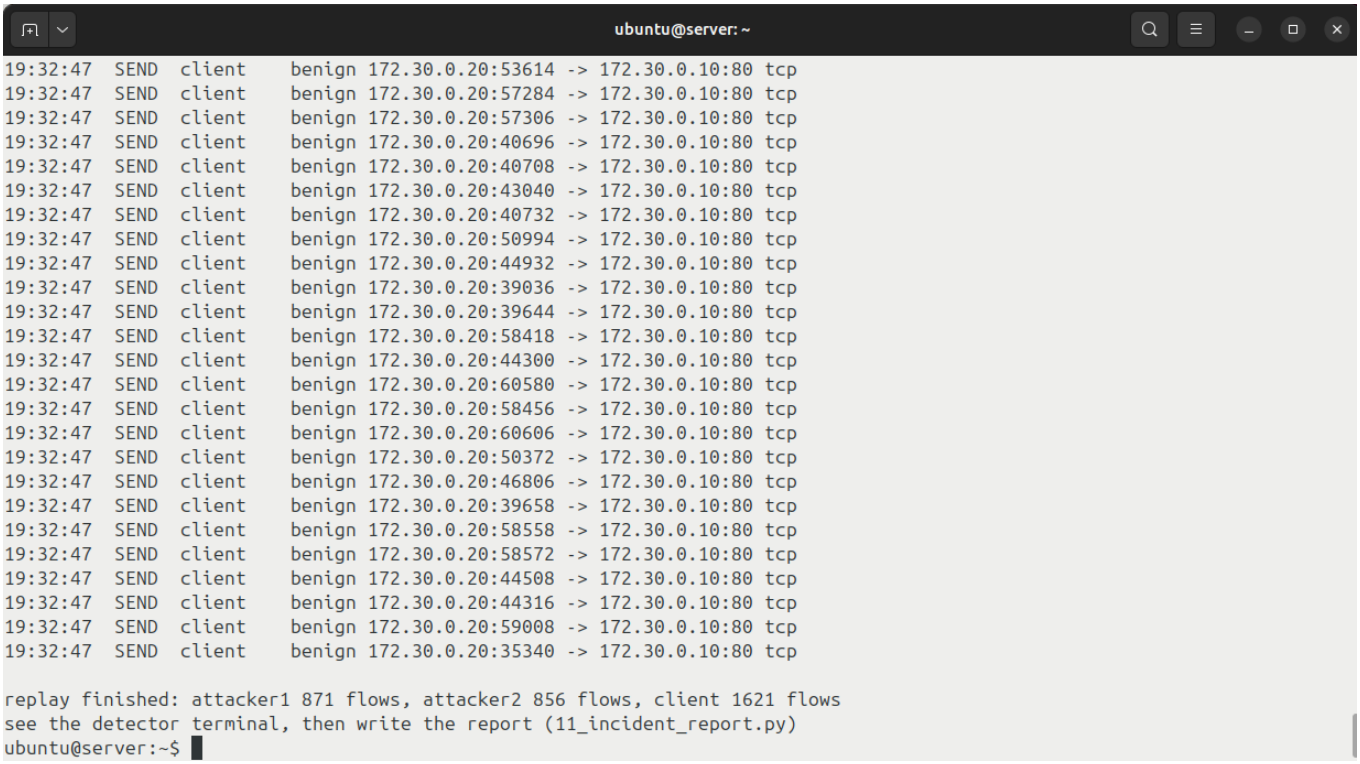


Figure 47: Terminal 2: the end of the replay

In terminal 1, the first alerts appear a few seconds after the attack starts. After three alerts from `172.30.0.66`, the address is blocked (Figure 48); when the window closes, the detector prints its summary (Figure 49):

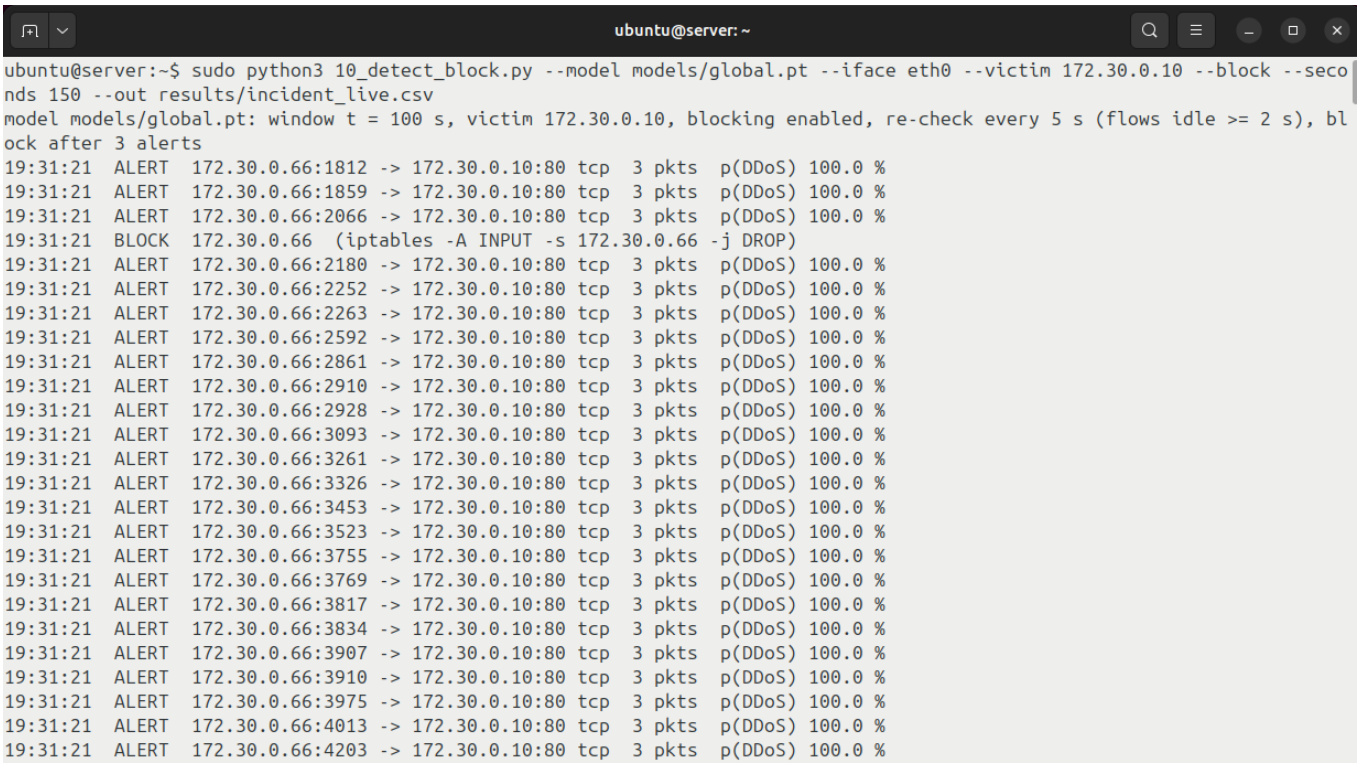


Figure 48: Terminal 1: the first alerts and the first block

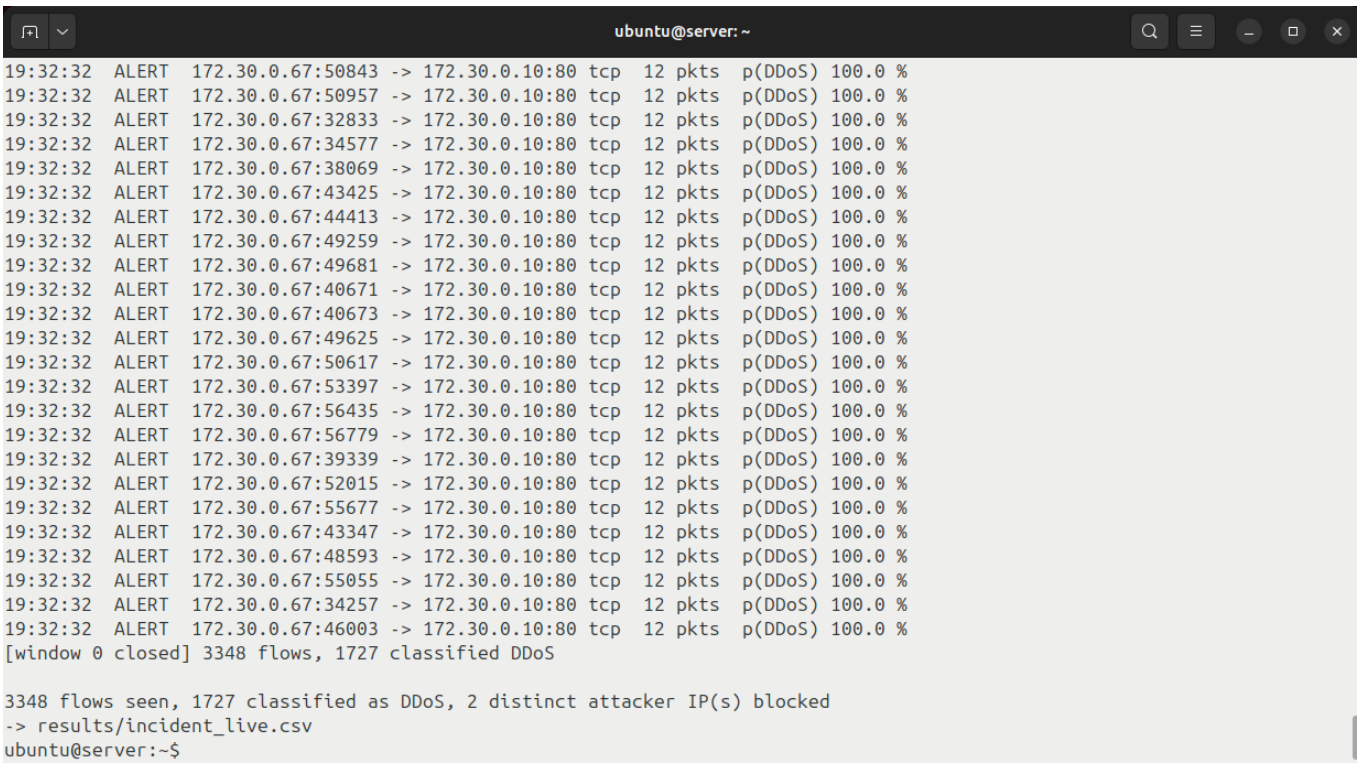


Figure 49: Terminal 1: the end of the detection window

9.5 Step 4: The Post-Incident Report

When the detector has finished, write the report:

```
python3 11_incident_report.py --log results/incident_live.csv --victim 172.30.0.10 \
    --attackers 172.30.0.66 172.30.0.67 --out results/incident_report.txt
```

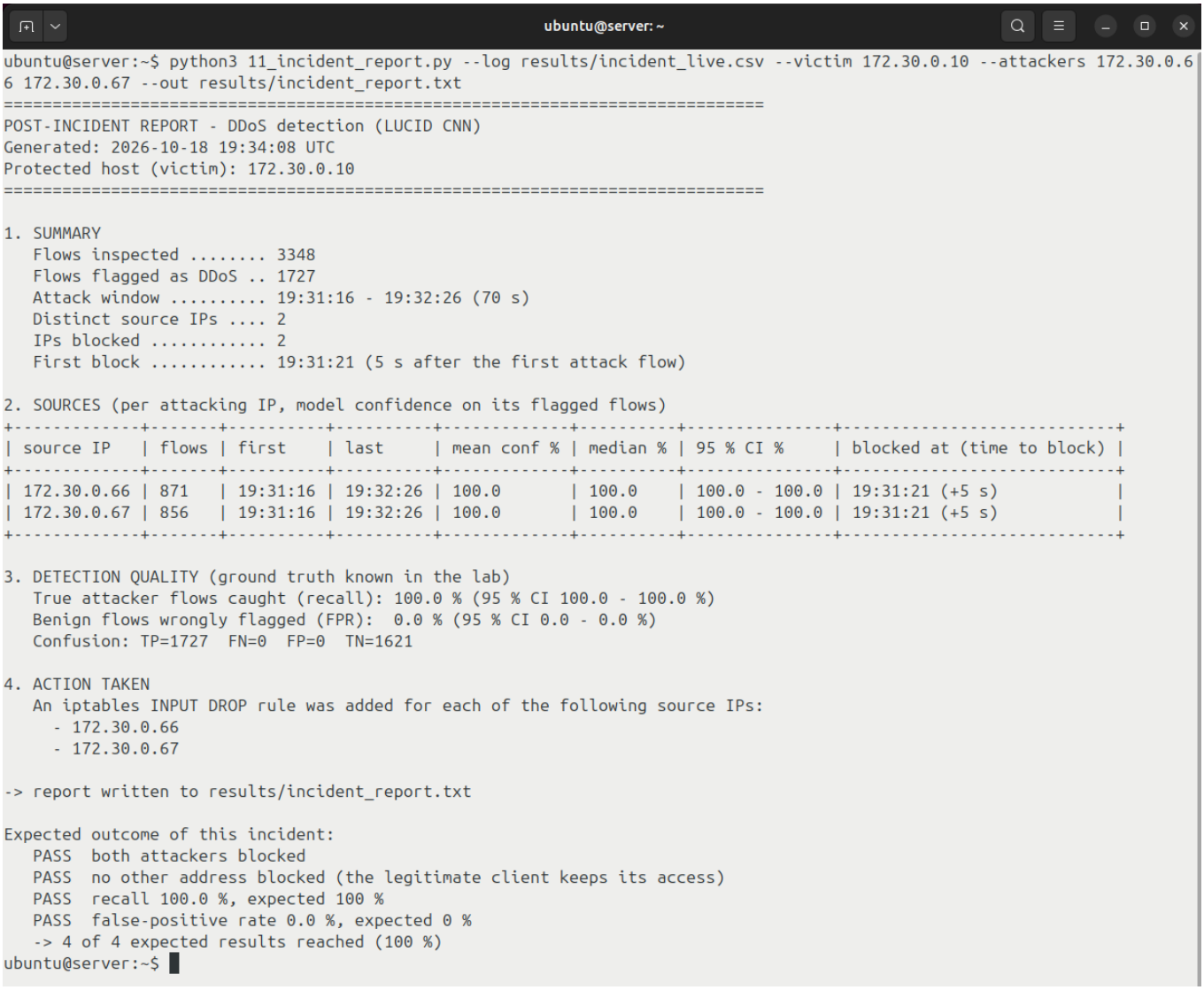


Figure 50: The post-incident report

Progress check: this step completes the `checkwork` goals `incident_report`, `attacker1_blocked`, `attacker2_blocked`, `only_attackers_blocked` (Section 12).
Below the report, the script checks the expected outcome of the incident: both attackers blocked, no other address blocked, recall 100 % and false-positive rate 0 %. Every check shows `PASS` or `FAIL`, and the last line gives the share of expected results you reached (Figure 50: 4 of 4, 100 %). These lines are only shown on the screen; they are not written to the report file. The report gives the attack window, every flagged address with the model's confidence and its 95 % confidence interval, the time it took to block each attacker, and the detection quality (recall and false-positive rate) compared with the known ground truth. It is saved in `results/incident_report.txt`.
Questions to think about before you write the report (Section 10):

- How many seconds passed between the first attack flow and the first block? What limits this time?
- Was the legitimate client blocked? Which detector settings protect it (see `--help`)?
- Would the model still detect the attack if the attackers used new addresses? Why?

10 Deliverable: The Post-Incident Report

After an incident, a security team documents what happened, how it was handled and what should change. As the analyst on duty, write a post-incident report on the attack you defended against. Use `results/incident_report.txt` and the figures and tables of `09_plot.py` as your evidence. Your report must contain the following parts:

1. **Summary:** What happened, when, which host was attacked, and the outcome.
2. **Timeline:** The first attack flow, the first alert, the time each attacker was blocked and the end of the attack.
3. **Attack sources and attack types:** Which IP addresses attacked, how many flows each sent, and which kinds of attack they used (see the hint below).
4. **Why the traffic was classified as an attack:** Which properties of the flows the model relies on. Remember that the model never sees IP addresses or ports (Experiment 2): the decision is based on how the traffic behaves, not on who sends it. Use the permutation importance of your model (Experiment 5) as evidence.
5. **Confidence:** The model's mean and median confidence on the flagged flows of every address with its 95 % confidence interval, and the test-split metrics of your model with their confidence intervals (Figure 36 and Table 3, run `global`; on the lab network alone, Table 8, run `cic + lab (test lab)`). Explain what the intervals mean.
6. **Detection quality:** Recall and false-positive rate with their confidence intervals. Was any legitimate host flagged or blocked?
7. **Actions taken:** The firewall rules that were added and how long it took to block each attacker. Check them with `sudo iptables -L INPUT -n`.
8. **Lessons learned and recommendations:** What would you change? For example, the number of alerts before blocking (`--min-alerts`), the window length, or what happens if the attackers move to new addresses. Would a classical threshold rule have stopped this attack, and would it still work against a botnet (Experiment 4)? Would a model trained only on a public dataset have stopped it (Experiment 6)?

Hint: identifying the attack types. The detection log lists every flow with its source address, number of packets and decision. Count the flagged flows per address and size:

```
awk -F, '$9==1 {print $3, $7}' results/incident_live.csv | sort | uniq -c
```

In this lab the output looks like this:

```
441 172.30.0.66 12
430 172.30.0.66 3
419 172.30.0.67 12
  1 172.30.0.67 13
436 172.30.0.67 3
```

Both attackers send two kinds of flows. Look at a few of them in the replayed files to find out what they are, for example:

```
tshark -r replay/attacker1.pcap -Y "tcp.flags.reset==1" | head
tshark -r replay/attacker1.pcap -Y "http.request" | head
```

A flow of three packets (SYN, SYN-ACK, reset) never completes the TCP handshake, which is the pattern of a **SYN flood**; a longer flow that carries an HTTP request is part of an **HTTP flood**. State in your report which evidence led you to this conclusion.

11 Terminating the Laboratory

After completing the analysis, stop the lab from the Labtainer terminal (directory `~/labtainer/labtainer-student`):

```
stoplab lucid-ddos
```

After termination, the folder `lucid-ddos` is created inside the `labtainer_xfer` folder on the virtual machine desktop (Figure 51). It contains the archive with your work, which is submitted for grading.

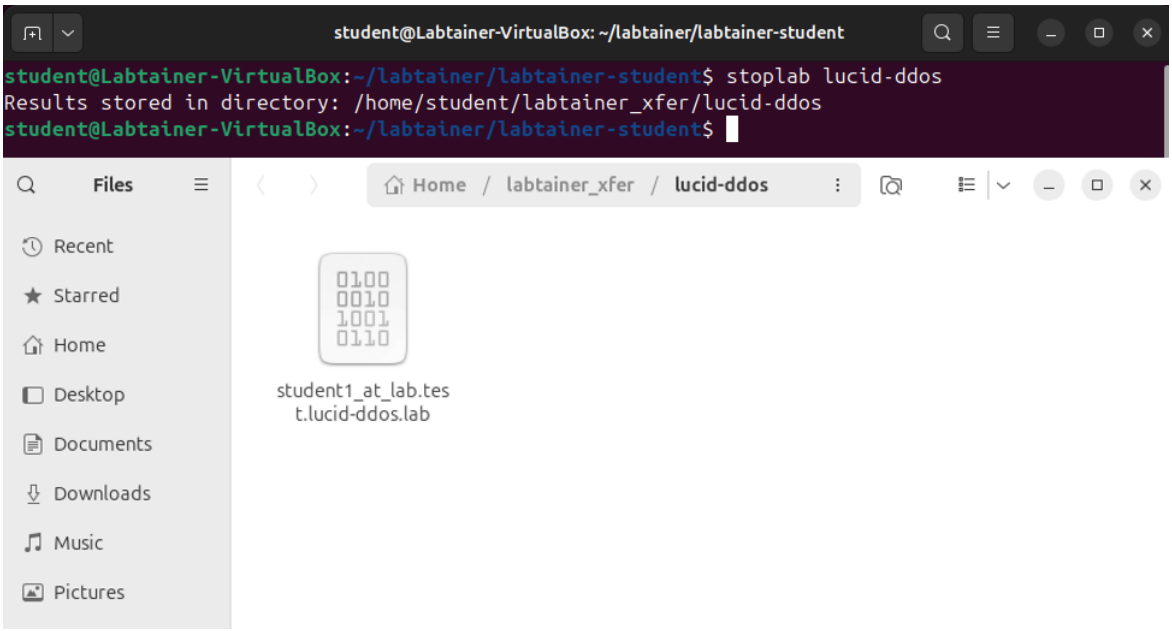


Figure 51: Stopping the lab and the results folder

12 Self-Assessment Before Submitting

Check your own work in two ways. First, the automatic check shows which parts of the lab are completed; you can run it after every step and a last time before you submit; second, your post-incident report and your answers to the analysis questions show whether you understood the results.

12.1 Automatic Check of Your Work

Labtainer can show which parts of the lab are already complete. You can run the check at any time, also while the lab is running, for example right after a step to confirm that its goal is met (the *Progress check* notes after the steps name the goals). Run in the Labtainer terminal (directory `~/labtainer/labtainer-student`):

```
checkwork lucid-ddos
```

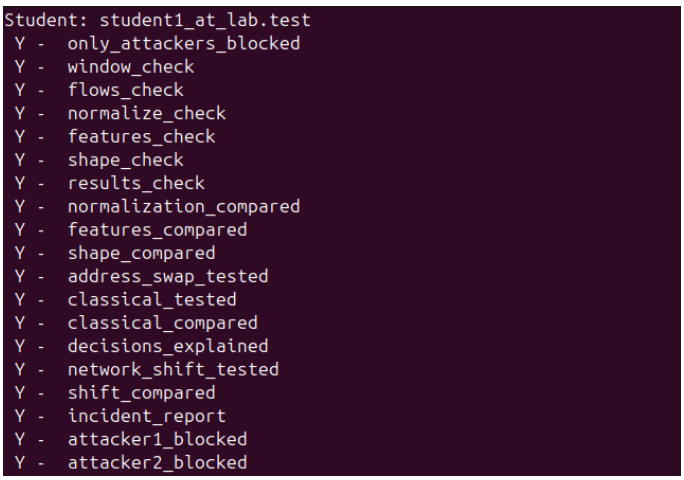


Figure 52: The automatic check of a complete lab: every goal shows Y

Every line is one goal; `Y` means the goal is met and `N` means it is not yet met (Figure 52). Table 10 lists the goals and what each of them requires.

Goal	**Met when**
`window_check`, `flows_check`, `normalize_check`, `features_check`, `shape_check`	Every preprocessing step passed all rules of `check.py` (Section 6)
`results_check`	All 23 runs match the reference results (Section 8.7)
`normalization_compared`, `features_compared`, `shape_compared`	The comparison figures were drawn with `09_plot.py` (Section 8.8)
`address_swap_tested`	A model was tested on the address-swapped split `test_newaddr` (Experiment 2)
`classical_tested`, `classical_compared`	The classical rule was run against the distributed attack and compared (Experiment 4)
`decisions_explained`	`15_explain.py` explained a model (Experiment 5)
`network_shift_tested`, `shift_compared`	The model trained on the CIC-DDoS2019 sample was tested on the lab network, and the comparison figure was drawn (Experiment 6)
`incident_report`	The post-incident report was written by `11_incident_report.py` (Section 9.5)
`attacker1_blocked`, `attacker2_blocked`	Both attackers were blocked in the live test (Section 9)
`only_attackers_blocked`	Exactly the two attackers were blocked; the legitimate client was not

Table 10: The goals checked by `checkwork`

Important: Labtainer only records commands that you run in the lab terminals. If a goal stays empty although you did the step, run the command again in the server terminal and run `checkwork` again. The check takes up to about a minute and can be repeated as often as you like; run it a last time after stopping the lab (Section 11), before you submit.

12.2 Reviewing Your Own Results

The automatic check only shows that a step was done, not that you understood it. Before you submit, go through your results once more:

- Can you explain every result in Tables 3–8 in your own words, and why it differs between the runs of the same experiment?
- Have you answered the analysis questions of Section 8.8 and Section 9.5 with numbers from your own runs?
- Does your post-incident report contain all eight parts of Section 10, and is every claim in it backed by a figure, a table or a line of `results/incident_report.txt`?

Congratulations!
Through this laboratory exercise, you have built a complete DDoS detection pipeline from raw packets, compared normalization, feature selection and shaping choices with confidence intervals, tested how a model carries over to another network, and defended a live server against an attack, from detection to the post-incident report.

References

[1] R. Doriguzzi-Corin, S. Millar, S. Scott-Hayward, J. Martínez-del-Rincón and D. Siracusa, "LUCID: A Practical, Lightweight Deep Learning Solution for DDoS Attack Detection," *IEEE Transactions on Network and Service Management*, vol. 17, no. 2, pp. 876–889, 2020. `https://doi.org/10.1109/TNSM.2020.2971776`

[2] A. Wang, W. Chang, S. Chen and A. Mohaisen, "Delving Into Internet DDoS Attacks by Botnets: Characterization and Analysis," *IEEE/ACM Transactions on Networking*, vol. 26, no. 6, pp. 2843–2855, 2018. `https://doi.org/10.1109/TNET.2018.2874896`

[3] Cloudflare, "2025 Q4 DDoS threat report: A record-setting 31.4 Tbps attack caps a year of massive DDoS assaults," February 5, 2026. `https://blog.cloudflare.com/ddos-threat-report-2025-q4/`

[4] A. Shiravi, H. Shiravi, M. Tavallaee and A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark datasets for intrusion detection," *Computers & Security*, vol. 31, no. 3, pp. 357–374, 2012.

[5] I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 108–116, 2018.

[6] Canadian Institute for Cybersecurity, "CSE-CIC-IDS2018 dataset," 2018. `https://www.unb.ca/cic/datasets/ids-2018.html`

[7] R. Doriguzzi-Corin, "LUCID: Lightweight, Usable CNN in DDoS Detection," source code and sample dataset (folder `sample-dataset`), Apache License 2.0. `https://github.com/doriguzzi/lucid-ddos`

[8] I. Sharafaldin, A. H. Lashkari, S. Hakak and A. A. Ghorbani, "Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy," in *Proc. 53rd International Carnahan Conference on Security Technology (ICCST)*, 2019. `https://www.unb.ca/cic/datasets/ddos-2019.html`